Budapesti Műszaki és Gazdaságtudományi Egyetem

Villamosmérnöki és Informatikai Kar

Távközlési és Mesterséges Intelligencia Tanszék

Tudásgráf-építés műszaki rajzokból és adatlapokból skálázható Q&A rendszerhez

Knowledge Graph Construction from Engineering Drawings and Datasheets for a Scalable Q&A System

Tudományos Diákköri Konferencia dolgozat

Szerzők: Szabó Zsombor István, Kiszely Benedek László

Témavezető: Dr. Hollósi Gergely László

Budapest, 2026

Kivonat

Egy ipari üzemet sokféle dokumentum ír le: a csővezeték- és műszerezési rajzok (P&ID) azt, hogyan kapcsolódnak egymáshoz a berendezések, a gyártói adatlapok azt, mire képes az egyes eszköz. Egy mérnöki kérdés gyakran mindkettőt igényli, például egy szivattyú utáni szelep megengedett nyomása. A nagy nyelvi modellek ilyen kérdésekre csak akkor válaszolnak megbízhatóan, ha a dokumentumok szerkezete (melyik cső hová vezet, melyik érték milyen feltétellel érvényes) megmarad. Ezt egy tudásgráf, vagyis kapcsolatokat tároló adatbázis tudja megőrizni.

A munka két részből épül fel. A rajzoldali rész az egész üzem rajzlapjaiból épít tudásgráfot, amelyben a lapok határán megszakadó vezetékek újra összekapcsolódnak, és egy kétlépcsős rendszerrel (előbb egy adatbázis-lekérdezés, szükség esetén gráfbejáró eszközöket hívó modell) válaszol rajta. Az adatlapoldali rész egy-egy eszköz PDF-adatlapjából épít gráfot, amelyben minden érték mellett megmarad a mértékegysége, az érvényességi feltétele és a forrásoldala. A dolgozatban a két részt összekapcsoljuk: a rajz elemeit az azonosítójuk (tag-jük) alapján az adatlapgráfokhoz kötjük egy közös adatbázisban, és három kérdés-válaszoló módszert hasonlítunk össze: az összevont gráfon dolgozó rendszert (M1), a két meglévő rendszert külön hívó irányítót (M2) és egy gráf nélküli alapmódszert (M3).

A módszereket egy négylapos prototípus-üzemen építettük fel, a közös rendszert egy 30 és egy 50 lapos szintetikus üzemen hangoltuk, majd befagyasztva egy 100 és egy 1000 lapos, a hangolás során nem használt üzemen mértük meg, üzemenként 60 kérdésen és 12, több adatlapot összevető kérdésen, két nyelvi modellel (GLM-5.3 Flash és GPT-6 Luna). Mindkét tesztüzemen az irányító volt a legjobb a GLM-5.3 Flash modellel (51, illetve 48 helyes válasz a 60-ból), az összevont gráfon dolgozó módszer ugyanezzel a modellel 50, illetve 43 helyes választ adott; a kettő különbsége nem szignifikáns. A gráf nélküli alapmódszer a 100 lapos üzemen kevesebb helyes választ adott, a GPT-6 Luna modellel tízszer annyiba került kérdésenként, mint az irányító, az 1000 lapos rajz pedig már nem fér el a modellek kontextusában, így ott nem is futtatható. A gráfos módszerek kérdésenkénti költsége a tízszer nagyobb rajzon legfeljebb a kétszeresére nőtt. Az eredmények szintetikus üzemekre és modell által írt referenciaválaszokra érvényesek.

Kulcsszavak: tudásgráf, GraphRAG, nagy nyelvi modellek, P&ID, műszaki adatlapok, kérdés-válaszolás, ipari dokumentáció.

Abstract

An industrial plant is described by many kinds of documents: piping and instrumentation diagrams (P&IDs) show how the equipment is connected, vendor datasheets say what each device can do. Engineering questions often need both, for example the allowed pressure of the valve after a pump. Large language models answer such questions reliably only if the structure of the documents (which pipe leads where, which value holds under which condition) is kept. A knowledge graph, a database that stores these relations, can keep it.

The work has two parts. The drawing part builds a knowledge graph from all sheets of a plant, reconnecting the lines cut at sheet borders, and answers over it with a two-stage system (a database query first, then a model that calls graph-walking tools if needed). The datasheet part turns one device's PDF datasheet into a graph in which every value keeps its unit, its validity condition and its source page. In this thesis we connect the two: the drawing items are linked to the datasheet graphs by their identifiers (tags) in one database, and we compare three question-answering methods: a system over the merged graph (M1), a router that calls the two existing systems separately (M2), and a graph-free baseline (M3).

We built a first prototype on a four-sheet plant, tuned the joint system on two synthetic plants of 30 and 50 sheets, froze it, and measured it on a 100-sheet and a 1000-sheet plant not used during tuning, on 60 questions and 12 questions that compare several datasheets per plant, with two language models (GLM-5.3 Flash and GPT-6 Luna). On both test plants the router with GLM-5.3 Flash was best (51 and 48 of 60 correct); the method over the merged graph gave 50 and 43 correct answers with the same model, and the difference is not significant. On the 100-sheet plant the graph-free baseline gave fewer correct answers and, with GPT-6 Luna, cost ten times as much per question as the router; the 1000-sheet drawing no longer fits into the models' context, so the baseline cannot run there. The per-question cost of the graph-based methods at most doubled on the ten times larger drawing. The results hold for synthetic plants and model-written reference answers.

Keywords: knowledge graph, GraphRAG, large language models, P&ID, technical datasheets, question answering, industrial documentation.

Tartalomjegyzék

Kivonat ... 2
Abstract ... 3
Tartalomjegyzék ... 4
1. Bevezetés ... 7
1.1 Kutatási kérdések ... 8
1.2 Hozzájárulások ... 8
1.3 Munkamegosztás ... 8
1.4 A dolgozat felépítése ... 9
2. Háttér és kapcsolódó munkák ... 10
2.1 P&ID-k és gyártói adatlapok ... 10
A P&ID mint dokumentum ... 10
A DEXPI és a pyDEXPI ... 11
A gyártói adatlap ... 11
Szabványok az adatlapi tények ábrázolására ... 11
Tulajdonsággráf és RDF ... 12
Feltételes, n-áris tények ... 12
2.2 Kérdés-válaszolás P&ID-ken ... 12
Visszakeresés a rajz gráfján ... 12
A rajzok gráffá alakítása ... 13
Üzemi méret ... 13
2.3 Kérdés-válaszolás műszaki dokumentumokon ... 14
2.4 GraphRAG és visszakeresési stratégiák ... 15
Mikor segít a gráf? ... 15
Gráfeszközöket hívó ágensek ... 15
Kaszkád és önjelzés ... 15
2.5 Összevetés ... 16
2.6 A hiányzó lépés ... 16
3. Adatok és feladat ... 18
3.1 A kis példaüzem (az első prototípus) ... 18
3.2 A fejlesztési és a tesztüzemek ... 19
3.3 Adatlapok ... 20
3.4 A kérdéssor ... 21
3.5 Pontozás ... 21
4. Az összevont tudásgráf ... 23
4.1 A rajzoldali réteg ... 23
Az üzemi réteg és a vonalnézet ... 23
4.2 Az adatlapoldali réteg ... 24

Az adatlapréteg a Neo4j-ben ... 24
4.3 Az összekötés ... 25
4.4 Betöltés ... 26
5. Kérdés-megválaszolási módszerek ... 27
5.1 Közös építőelemek ... 27
A rajzoldali kaszkád ... 27
Az ágens primitívjei ... 28
Az adatlapoldali A2 ... 29
5.2 M1 — egy gráf, egy lekérdezés ... 29
5.3 M2 — router ... 30
5.4 M3 — gráf nélküli baseline ... 31
5.5 Hangolás a fejlesztési üzemeken ... 31
5.6 A módszerek várható erősségei és gyengeségei ... 31
M1 ... 31
M2 ... 32
M3 ... 32
5.7 Költség és mérés ... 32
6. Kiértékelés ... 33
6.1 Beállítás ... 33
6.2 Az első prototípus a kis üzemen ... 33
6.3 Hangolás a fejlesztési üzemeken ... 33
6.4 A gráf nélküli módszer az 1000 lapos üzemen ... 34
6.5 Eredmény a két tesztüzemen ... 34
6.6 Költség ... 36
6.7 Több adatlapos kérdések ... 36
6.8 Hibaelemzés ... 37
7. Demó és esettanulmány ... 38
7.1 A demó felépítése ... 38
7.2 Két kérdés végigkövetve ... 38
7.3 Tanulságok ... 39
8. Összefoglalás ... 40
8.1 Válaszok a kutatási kérdésekre ... 40
8.2 Korlátok ... 40
8.3 Továbblépés ... 41
Irodalomjegyzék ... 42
Függelék ... 44
A. Példakérdések családonként (dev halmaz) ... 44
B. Az M2 router rendszerüzenete ... 44
C. Az M1 Cypher-szintjének sémakiegészítése (eleje) és az új primitív leírása ... 45

D. A két forrásfolyamat részletei ... 45
A plantgraph üzemgenerátora ... 45
A plantgraph lapokra bontója és a lapközi csatlakozók ... 46
A plantgraph feloldója ... 46
Az adatlapok PDF-jei ... 47
A plantdocs folyamatának lépései ... 47
A plantdocs célformátuma ... 48
A plantdocs RDF-modellje ... 49
E. A módszerek részletei ... 50
Az ágens primitívjei ... 50
Az adatlapoldali A2 lépései ... 51
Az M1 sémakiegészítésének felépítése ... 52
Az M2 router protokollja ... 52
Útvonalak a tesztüzemeken ... 52
F. További demó-képernyőképek ... 52
Két kérdés végigkövetve ... 52
Rajzi útvonal egyetlen lekérdezéssel (M1) ... 54
Több adatlapos kérdés (M2) ... 55
Két további kérdés képernyőkép nélkül ... 56
G. A kis üzem mérései ... 56
Fejlesztési iterációk ... 56
Végső mérés a teszthalmazon ... 57
Költség és késleltetés ... 58
Útvonalak ... 59
Összetett, több adatlapos kérdések ... 59
Hibaelemzés ... 60

1. Bevezetés

Egy ipari üzemet sokféle dokumentum ír le. A csővezeték- és műszerezési rajzok (angol rövidítéssel P&ID) megmutatják, hogyan kapcsolódnak egymáshoz a berendezések, szelepek és műszerek; a gyártói adatlapok megadják, mire képes az egyes eszköz. A rajzon minden elemnek egyedi azonosítója, tag-je van (például P-101 egy szivattyú), és ez köti össze a rajzot az eszköz adatlapjával. Egy üzemeltető vagy tervező mérnök kérdései gyakran mindkét dokumentumfajtát érintik: „Mekkora a megengedett legnagyobb nyomás azon a szelepen, amely a P-101 szivattyú nyomóoldalát elzárja?" A válaszhoz előbb a rajzon kell megtalálni a szelepet (a vezeték akár egy másik rajzlapon is folytatódhat), majd a tag-je alapján az adatlapját, és abban a megfelelő feltételhez tartozó értéket. Ma ezt kézzel végzik.

A nagy nyelvi modellek (LLM) kézenfekvő eszköznek tűnnek, de az üzemi dokumentáció nehéz terep számukra. A lényeges információ gyakran nem egyetlen elemben van, hanem a kontextusban: egy táblázat fejlécében, egy értéket korlátozó lábjegyzetben, vagy egy lapközi csatlakozóban, amely egy csővezetéket egy másik lapon folytat. Ezt a szerkezetet egy tudásgráf teheti explicitté: olyan adatbázis, amelyben a dolgok csúcsok, a köztük lévő kapcsolatok élek, és amelyből a modell a kérdéshez tartozó részt visszakeresve válaszol (az angol szakirodalomban: GraphRAG). Ez azonban csak akkor működik, ha a gráf megőrzi a kontextust.

A munka két részből épül fel, mindkettő az egyik szerző egyéni munkája.

- **A rajzoldali rész** (Kiszely Benedek László rendszere, a továbbiakban **plantgraph**) az egész üzemet leíró tudásgráfot épít a rajzlapokból: a berendezések és műszerek csúcsok, a csövek és jelvezetékek élek, és a lapok határán megszakadó vezetékek újra összekapcsolódnak. Erre a gráfra építve egy kétlépcsős rendszer, a **kaszkád** (lépcsőzetes módszer) válaszol a kérdésekre: először egyetlen adatbázis-lekérdezéssel próbálkozik, és ha ez nem elég, egy gráfbejáró eszközöket hívó LLM-ágens veszi át a kérdést. A rész fő kérdése, hogy ez a megközelítés több száz, akár ezer lapos üzemen is működik-e.
- **Az adatlapoldali rész** (Szabó Zsombor István rendszere, a továbbiakban **plantdocs**) egy-egy eszköz PDF-adatlapjából épít tudásgráfot, amelyben minden érték mellett megmarad a mértékegysége, az érvényességi feltétele (például „csak 100 °F alatt") és a forrásoldala, vagyis az adatlap PDF-jének az az oldala, ahol az érték nyomtatva szerepel, hogy a válasz az eredeti dokumentumban ellenőrizhető legyen. A feltételt azért kell megőrizni, mert egy feltétel nélküli érték nem hiányos, hanem hibás. A gráf nemzetközi ipari szabványok szókészletére épül, ezeket a 2. fejezet mutatja be. A plantdocs két lekérdezési módszert hasonlított össze: az **A1** a gráf RDF-formáját SPARQL-lekérdezéssel kérdezi, az **A2** a Neo4j-adatbázisba töltött gráfra egy nyelvi modell által írt Cypher-lekérdezéssel és vektoros (jelentés szerinti) kereséssel válaszol. Ebben a munkában az A2-t használjuk.

A két rendszer felépítését a 4. és az 5. fejezet foglalja össze, részletes leírásuk a Függelék D-ben található.

Ebben a dolgozatban a két részt összekapcsoljuk: a rajzi gráf elemeit a tag-jük alapján az adatlapgráfokhoz kötjük, és megmérjük, mennyire jól válaszolnak a rendszerek a két dokumentumfajtát együtt igénylő kérdésekre. Az első prototípus egy kis, négylapos üzemen készült; a közös rendszert ezután két fejlesztési üzemen (30 és 50 lap) hangoltuk, végül a

hangolt és befagyasztott rendszert két olyan üzemen mértük meg (100 és 1000 lap), amelyeket a hangolás során nem használtunk.

1.1 Kutatási kérdések

- **RQ1.** Megválaszolhatók-e a rajzot és az adatlapot összekötő kérdések egyetlen összevont tudásgráfon (M1), és hogyan viszonyul ennek pontossága egy olyan irányítóéhoz (router, M2), amely a két rész meglévő rendszerét külön hívja, illetve egy gráf nélküli alapmódszeréhez (baseline, M3)?
- **RQ2.** Hogyan változik a módszerek pontossága és költsége, ha az üzem mérete nő? A módszereket a 30 és 50 lapos fejlesztési üzemen hangoltuk, és a hangolt, befagyasztott módszereket a 100 és 1000 lapos tesztüzemen mértük, további változtatás nélkül.
- **RQ3.** Mely kérdéscsaládokon segít a gráf, és milyen hibák jellemzők az egyes módszerekre?

1.2 Hozzájárulások

1. Öt példaüzem ugyanazzal a generátorral: egy négylapos prototípus-üzem (26 tagelt elem), két fejlesztési üzem pontosan 30 (206) és 50 lappal (378), valamint két tesztüzem pontosan 100 (772) és 1000 lappal (7854). Az üzemek elemeihez program rendeli hozzá hét eszköz gyártói adatlapját. Az, hogy több száz vagy ezer rajzi elemhez csak hét adatlap tartozik, nem ellentmondás: egy adatlap egy eszköztípust (egy gyártó katalógusában szereplő modellt) ír le, és az azonos típusú rajzi elemek (például az összes centrifugálszivattyú) ugyanazt az adatlapot kapják, ahogy egy valódi üzemben is ugyanaz a szivattyúmodell sok helyen be van építve. A rajzok szintetikusak, a hét adatlap viszont valódi gyártói PDF (3. fejezet; a fájlok helyét a Függelék D adja meg).
2. Az összevont tudásgráf: a rajzi üzemi réteg és hét eszköz adatlapgráfja egy Neo4j-adatbázisban, egy tagalapú kapcsolattal összekötve, egy paranccsal újratölthetően.
3. Három kérdés-válaszoló módszer a két rész meglévő kódjára építve: a rajzoldali rendszer kiegészítése az adatlapréteggel és egy új adatlap-eszközzel (M1), egy irányító a két rendszer fölött (M2) és egy gráf nélküli alapmódszer (M3).
4. Üzemenként egy 60 kérdésből álló, négy családra bontott kérdéssor, és ezen felül 12 olyan kérdés, amelyhez több adatlapot kell összevetni. A rajzra vonatkozó kérdések referenciaválaszát program számolja ki a generátor megoldókulcsként szolgáló üzemgráfjából; az adatlapi kérdések referenciaválaszát egy nyelvi modell írta az adatlap PDF-jéből, és ezeket ember nem ellenőrizte; a kettőt kombináló kérdések referenciáját program rakja össze ezekből (3. fejezet). Ehhez tartozik a három módszer mért összehasonlítása két modellel a két tesztüzemen (az 1000 lapos üzemen a gráf nélküli alapmódszer nélkül, mert annak promptja ott nem fér a modellek kontextusába).
5. Egy webes demó, amely bármely módszerrel válaszol, és megmutatja az útvonalat, a lekérdezéseket és az adatlapi forrásoldalakat.

1.3 Munkamegosztás

A rajzoldali rendszer (generátor, lapokra bontás, üzemi gráf, kaszkád) Kiszely Benedek, az adatlapoldali rendszer (feldolgozó folyamat, ISO 15926 gráf, A2-lekérdezés) Szabó Zsombor egyéni munkájának eredménye; a kódjukat nem módosítottuk. Az összevonást, a kérdéssort, a három módszert, a mérést és a demót közösen készítettük.

1.4 A dolgozat felépítése

A 2. fejezet a hátteret és a kapcsolódó munkákat foglalja össze. A 3. fejezet bemutatja a példaüzemeket (a prototípus-üzemet, a két fejlesztési és a két tesztüzemet), az adatlapokat és a kérdéssort, a 4. fejezet az összevont tudásgráfot, az 5. fejezet a három módszert. A 6. fejezet a mért eredményeket, a 7. fejezet a demót és egy esettanulmányt, a 8. fejezet a következtetéseket és a korlátokat tárgyalja. A Függelék a példakérdéseket (A), a két módszer promptjainak részleteit (B, C), a két forrásfolyamat részletes leírását (D), a módszerek lépéseinek részleteit (E), további demó-képernyőképeket (F) és az első prototípus kis üzemen végzett méréseinek teljes leírását (G) tartalmazza.

2. Háttér és kapcsolódó munkák

Ebben a fejezetben összefoglaljuk a dokumentumokat, a szabványokat és a nyelvimodell-alapú kérdezési módszereket, amelyekre a dolgozat épül, majd egy táblázatban vetjük össze a kapcsolódó munkákat.

2.1 P&ID-k és gyártói adatlapok

A P&ID mint dokumentum

A csővezeték- és műszerezési rajz (P&ID) az üzem központi tervdokumentuma: megmutatja, mely berendezések, szelepek és műszerek vannak az üzemben, és hogyan kapcsolódnak egymáshoz[7,45].

A rajzi elemeket a tag azonosítja (pl. P-101, FT-202). A tag betűi az elem fajtáját és szerepét jelölik, a műszereknél az első betű a mért változót (F áramlás, P nyomás, L szint, T hőmérséklet), a további betűk a funkciót (például T távadó, IC szabályozó); a betűk a hurokszámmal együtt adnak egyedi azonosítót[1]. Az `FT-202` például így olvasható: F = a mért változó az áramlás, T = a műszer funkciója távadó, 202 = a szabályozókör (hurok) száma; ugyanennek a körnek a szabályozója `FIC-202`. A csővezetékek azonosítója, a vezetékszám szokás szerint névleges átmérőből, közegkódból, sorszámból, csőosztályból és szigetelésből áll[2]. Egy ilyen vezetékszám lehet például (csak szemléltető, kitalált példa) a `3"-CW-1012-CS1-N`: 3 hüvelyk névleges átmérő, CW (hűtővíz) közegkód, 1012-es sorszám, CS1 csőosztály, N (szigetelés nélkül). A példaüzemek tag-jei ugyanezt a betűrendszert követik, de a hurokszám helyett az egység és a sorszám áll bennük: a `P-1-3` az 1. egység harmadik szivattyúja (P = szivattyú), az `FT-2-2` a 2. egység második szabályozókörének áramlástávadója (F = áramlás, T = távadó).

Egy valódi üzem rajza sok lapra tagolódik. A laphatáron átmenő vezetéket lapközi csatlakozó (off-page connector) folytatja egy másik lapon: a lap szélén álló, feliratozott jel, amely megnevezi a másik lapot, és amelyhez a másik lapon egy párja tartozik. A rajzi szabályok szerint a többlapos rajzok vezetékvégei kölcsönösen hivatkoznak egymásra[21], és a rajzok közötti hivatkozó jelek jól strukturált rajzszámot hordoznak, amely akár reguláris kifejezésekkel is összeköthető[28]. A laphatár két különböző kapcsolatot vághat el. Az egyik egy cső: ekkor két csatlakozó jelzi, hogy a vezeték a másik lapon folytatódik. A másik egy elem: ugyanaz a berendezés több lapon is megjelenik, az egyik lapon teljes adataival, a többin csak a tag-jével. Mindkét esetben a gráf építőjének kell a két lap darabjait összekötnie, és ha ez elmarad, más-más hiba keletkezik. Ha a csatlakozópár nincs összekötve, a cső két fele nem kapcsolódik: az út megszakad, a csatlakozó párja nélkül marad, és ez egy egyszerű ellenőrzéssel észrevehető. Ha viszont a berendezés előfordulásait nem vonja össze egy elemmé, ugyanaz a berendezés két külön csúcsként kerül a gráfba: az egyik lapon bekötött csövek az egyik csúcshoz, a másikon bekötöttek a másikhoz tartoznak. Mindkét csúcsnak vannak kapcsolatai, ezért a gráf épnek látszik, nincs párosítatlan jel, amely a hibát elárulná. Egy, a berendezésről szóló kérdés (például „mi táplálja ezt a hőcserélőt?”) azonban csak az egyik csúcs kapcsolatait látja, így a válasz hiányos lesz, és ezt semmi nem jelzi.

A DEXPI és a pyDEXPI

A rajz digitális cseréjének szabványa a DEXPI, amely UML információs modell saját XML-szerializációval[8]. A rajzi szabvány (például az ISO 10628) azt mondja meg, hogyan néz ki egy szimbólum; az információs modell azt, milyen osztályok és attribútumok vannak; az ontológia ugyanezt formális logikával, globális azonosítókkal írja le. A DEXPI információs modell: OWL- vagy RDF-formája nincs, de az osztálydefiníciói az ISO 15926 4. részének referenciakönyvtárából származnak, így szemantikailag le van horgonyozva[8]. Egy DEXPI-modell egyetlen rajzot ír le; rajzkészletet, és azt, hogy két lapon lévő elem ugyanaz, nem modellez.

A DEXPI szemantikus változata épül: a DEXPI egyesület hírei szerint a TotalEnergies a BFO-hoz és az IDO-hoz igazított DEXPI-n dolgozik[9], a POSC Caesar Arrowhead-kézikönyve pedig a DEXPI életciklus-zónáit és tulajdonságmodelljét igazítja az IDO-hoz, egyelőre egyetlen, kézzel írt szivattyús példán[5]. Az OPC UA felé is van leképezés[31]. Ezért ebben a dolgozatban nem építünk P&ID-ontológiát; a rajzoldalon a DEXPI osztályneveit használjuk címkeként, ahogy a kapcsolódó munkák is.

A DEXPI Python-megvalósítása a pyDEXPI: a DEXPI adatmodell Pydantic-osztályai, Proteus XML betöltő és NetworkX-gráffá alakító export[37]. A ChatP&ID és elődje a pyDEXPI osztálynevét teszi a gráfcsúcsok címkéjébe[3,4]; Benedek generátora is pyDEXPI-modellt épít, ezért a példaüzem elemei is ezeket az osztályneveket viselik (`CentrifugalPump`, `CheckValve`).

A gyártói adatlap

A rajz csak a kapcsolatokat mutatja; az egyes eszközök műszaki jellemzőit (nyomás- és hőmérséklethatár, anyag, méret, opciók) a gyártói adatlapok tartalmazzák. Egy adatlap néhány oldalastól több mint száz oldalasig terjedő PDF táblázatokkal, lábjegyzetekkel, feltételekhez kötött értékekkel és diagramokkal.

Az adatlapi tény ritkán egyetlen szám. Egy szabályozószelep adatlapján egy hőmérsékleti határ csak egy adott testanyagra és csatlakozóméret-tartományra érvényes, egy lábjegyzet tovább szűkíti, és ugyanaz az érték két mértékegységben is ki van nyomtatva. Ha egy rendszer ebből csak a számot és az egységet őrzi meg, a válasza nem hiányos, hanem hibás lehet. A plantdocs végső mérésén éppen a feltételes kérdéseken volt nagy a két lekérdezési módszer különbsége: a 31 feltételes kérdésből az A1 21-re, az A2 28-ra, a 17 több feltételt kombináló kérdésből 7-re, illetve 15-re adott helyes választ.

Szabványok az adatlapi tények ábrázolására

Az adatlapi tények gépileg értelmezhető leírására több, egymásra épülő szabvány létezik. A fontosabbak:

- Az **IDO** (Industrial Data Ontology) az alapfogalmakat adja: fizikai tárgy, rész és egész, információ, érték és mértékegység. OWL 2 nyelven írt ontológia; korábban ISO/TR 15926-14 volt, ma ISO 23726-3 néven szabványosítják[19].
- A **PCA RDL** (a POSC Caesar szervezet referenciakönyvtára) a konkrét berendezés-, cső- és műszerosztályok és tulajdonságok listája[33].
- A **CFIHOS** azt írja le, milyen adatokat kell átadni egy üzem átadásakor (tag-ek, berendezésosztályok jellemzői, dokumentumadatok)[6].

- A **QUDT** a mértékegységeket és a mennyiségfajtákat írja le; berendezést nem[38].

Más szókészletek (OntoCAPE[30], SOSA/SSN[41], IOF[20], schema.org[39]) erre a célra nem alkalmasak: vagy régóta nem frissülnek, vagy nem írják le a folyamatüzemi berendezéseket. A plantdocs ezért az ISO 15926 vonalat választotta (4. fejezet). Ugyanerre épül egy friss munka is, amely a szelepek tervezési szabványait (például az ASME B16.34-et) IDO-alapú ontológiává alakítja[12]. Mivel a DEXPI osztályai is az ISO 15926-ból származnak, a rajzi és az adatlapi oldal ugyanarra a szemantikai alapra épül.

Tulajdonsággráf és RDF

Az RDF-ben minden állítás egy alany–állítmány–tárgy hármas, az azonosítók globálisak (IRI), és az élnek nincs saját tulajdonsága. A tulajdonsággráfban (labeled property graph, LPG; legismertebb megvalósítása a Neo4j) a csúcsoknak és az éleknek is lehetnek tulajdonságaik, az azonosítók lokálisak. A ChatP&ID szerzői mérlegelték az RDF-et, de az attribútumokban gazdag mérnöki adat miatt a tulajdonsággráfot választották[4], és Benedek rendszere is Neo4j-re épül. A plantdocs adatlapgráfja RDF-ben készül (mert az IDO OWL 2), de a neosemantics (n10s) bővítménnyel Neo4j-be is betölthető; ez a híd teszi lehetővé, hogy a két réteg egy adatbázisba kerüljön.

Feltételes, n-áris tények

Az adatlapi tények jelentős része n-áris: egy érték egy alanyról, egy tulajdonságról és több feltételről egyszerre szól. Az ilyen tények bináris hármasokra bontása elveszíti a feltételt, és szemantikai széttöredezéshez vezet[18]. Erre a hiper-relációs tudásgráfok és a hipergráfos RAG ad ábrázolást. A minősítők (qualifiers) javítják a linkpredikciót[42], a HyperGraphRAG pedig az n-áris tényeket hiperélekként tárolja[17]. A tárolás kérdése tehát elvben megoldott: ismert, hogyan lehet egy értéket a mértékegységével és a feltételeivel együtt egy gráfban rögzíteni. A nehézség a kiolvasás: a PDF a feltételt gyakran csak az elrendezésével fejezi ki, például egy táblázat fejlécével, egy lábjegyzettel vagy azzal, hogy egy érték egy diagram melyik tartományába esik, és ezt kell helyesen felismerni. Hogy a mi adatlapgráfunk hogyan tárolja a feltételeket, azt a 4. fejezet mutatja be.

2.2 Kérdés-válaszolás P&ID-ken

Visszakeresés a rajz gráfján

A legközelebbi előzmény a ChatP&ID[4]: DEXPI-rajzokat pyDEXPI-vel Neo4j-gráffá alakít, és egy iteratív ágens négy visszakeresési eszköz közül választ. A négy eszköz négy különböző választ ad arra, hogyan jut el a gráf releváns része a modellhez. A ContextRAG az egész (sűrített) gráfot szövegként a promptba teszi, keresés nélkül. A VectorRAG a kérdéshez jelentés szerint leghasonlóbb csúcsok leírását adja, beágyazásvektorok koszinusz-hasonlósága alapján. A PathRAG egy így talált kiinduló csúcsból lépésenként bejárja a szomszédokat. A CypherRAG-ban a modell a kérdést Cypher-lekérdezéssé fordítja, az adatbázis lefuttatja, és az eredmény kerül a promptba. Egyetlen lapon, 19 kérdésen a ContextRAG adta a legjobb eredményt; a szerzők szerint viszont ez a módszer többlapos rajzoknál nem skálázódik. Egy korábbi munka ugyanettől a csoporttól a sűrített gráfot GraphML-ként, visszakeresés nélkül adja a modellnek, egy folyamatábrán, három kérdésen

értékelve[3]. A legnagyobb közölt P&ID-kérdéskészlet a PIDQA-é: 500 szintetikus rajzlapon 64 000 programmal előállított kérdés-válasz pár, négy kategóriában (egyszerű és térbeli számlálás, térbeli kapcsolat, értékalapú kérdés). A rajzból képfelismeréssel tulajdonsággráf készül Neo4j-ben, a kérdést pedig egy LLM fordítja Cypher-lekérdezéssé; minden kérdés egyetlen lapra vonatkozik, lapokon vagy dokumentumokon átívelő kérdés nincs benne[15].

Raszteres rajzokon a Gadekar és társai által javasolt módszer előbb visszanyeri a folyamatgráfot, majd a látás-nyelvi modell a topológiai kérdésekre csak hét, tényt visszaadó gráfoperátoron keresztül válaszolhat[11]; a TopoPID-VQA kérdésein ez a megkötés jelentősen javítja a pontos egyezést, de egylapos, szintetikus rajzokon. A szoftveres előzmények között egy nyílt LangGraph-ágens hét általános gráfeszközzel kérdez DEXPI-fájlokon, kérdésenként egy fájlon[35], egy ipari esettanulmány pedig olyan Neo4j-tudásgráfot ír le, amely helyreállítja a lapok közötti kapcsolatokat, és fölötte egy ágens előre definiált gráflekérdező eszközökkel és szabad Cypherrel válaszol, kiértékelés közlése nélkül[25].

A rajzok gráffá alakítása

A rajzok digitalizálását mi nem végezzük el, de a kérdés-válaszolás bemenete ennek a kimenete. Egy transzformeres módszer a szimbólumokat és a kapcsolataikat együtt jósolja, és ehhez kiadta a PID2Graph adatkészletet, amelynek gráfjai tisztán geometriaiak, címkék és lapok közötti megoldókulcs nélkül[34,44]. Ipari rajzokból automatikus tudásgráf-építést[27], a pyDEXPI-gráfon szabályalapú javítást[40], és mesterséges rajzgenerálást[16] is javasoltak. A képfelismerés töredezett vonalszakaszait folytonos kapcsolati gráffá egyesítő módszer[36] és a digitalizálást tagkinyerésként és topológia-következtetésként, két lépésben, multimodális nyelvi modellel megoldó megközelítés[47] is a rajz gráfját állítja elő, amelyre a kérdés-válaszolás épül. Valódi, többlapos, nyilvános rajzsorozatból kevés van; az OPEN100 sorozat 12 lapja ezek közé tartozik[32].

Üzemi méret

A fenti munkák közös vonása, hogy egy-egy rajzlapot vizsgálnak, és a rajzon kívüli dokumentumokat (adatlapokat) nem kapcsolják be. Kiszely Benedek egyéni munkája (plantgraph) a rajzi kérdés-válaszolást üzemi méretben vizsgálja, véletlenszerűen generált, többlapos szintetikus üzemeken (a generátor véletlenszám-generátorának kezdőértéke, a seed, meghatározza az üzemet: ugyanaz a seed mindig ugyanazt az üzemet adja), mert nyilvános, a lapok közötti kapcsolatok megoldókulcsával ellátott többlapos rajzkészlet nincs. A kérdéseket és a referenciaválaszokat program állítja elő a generátor megoldókulcsként szolgáló üzemgráfjából, és a válaszokat is program pontozza (ugyanezt a pontozót használjuk mi is, 3. fejezet); a pontosság a helyesen megválaszolt kérdések aránya. A generátornak két változata van: egy egyszerűbb, első változat, és egy valósághűbb, későbbi változat, amelynek üzemeiben oszlopok, reaktorok, biztonsági szelepek és a berendezések között sok jelöletlen idom is van. Az alábbiakban minden számnál megadjuk, melyik változat üzemein mérték.

A gráf nélküli alapmódszer. A ContextRAG a teljes rajzot szövegként a promptba teszi. Az egyszerűbb generátor üzemein a pontossága 1-ről 30 lapra 0,824-ről 0,658-ra csökkent, miközben a kaszkádé 0,941-ről 0,947-re változott. A valósághűbb generátor üzemein, egy

1, 3 és 11 lapos rajzokon végzett mérés-sorozatban a kaszkád pontossága 0,957 / 0,885 / 0,909, a ContextRAG-é 0,686 / 0,654 / 0,614 volt. A ContextRAG költségét a forrás a kaszkád költségéhez viszonyítva közli: kérdésenként az egyszerűbb üzemeken 1 lapon 1,8-szer, 30 lapon 41,5-ször, a valósághűbb üzemeken 1 lapon 3,79-szer, 11 lapon 21,57-szer annyiba kerül, mint a kaszkád. A szövegként leírt rajz egy bizonyos méret fölött már nem fér el a modell promptjában: az egyszerűbb generátor üzemein ez körülbelül 143, a valósághűbbén körülbelül 62 lap; a két szám tehát a két generátorváltozathoz tartozik.

A kaszkád 100 és 1000 lapos üzemen. Az egyszerűbb generátor üzemein a kaszkád a lezárt teszten 100 lapos üzemen 0,937, 1000 lapos üzemen 0,917 pontosságot ért el. A lezárt teszt 460 kérdése olyan üzemekre vonatkozott, amelyeket a fejlesztés során egyszer sem használtak: a generátor új seedekkel állította elő őket, a kérdéseket és a referenciaválaszokat pedig program készítette ezeknek az üzemeknek a megoldókulcsából, így a rendszer ezeket a rajzokat korábban nem láthatta. A valósághűbb generátor üzemein az egyszerűbb üzemekre fejlesztett rendszer gyengébb, ezért Benedek ezekre az üzemekre külön hangolt és befagyasztott egy kaszkádot; ezt használjuk mi is (5. fejezet). Két, a hangolás során nem használt üzemen (kb. 100 és kb. 1000 lap, egyenként 163 kérdés) a hangolt kaszkád 154, illetve 156, a hangolás előtti 145, illetve 144 helyes választ adott. A lezárt teszten (üzemenként 440 kérdés) a hangolt kaszkád kb. 100 lapon 401 (0,911), kb. 1000 lapon 392 (0,891) helyes választ adott. A −0,020-es különbség nem szignifikáns (a 95%-os konfidenciaintervallum −0,059 és +0,018 között van, tehát a nullát is tartalmazza), vagyis a tízszer nagyobb üzemen a pontosság nem romlott mérhetően. A kérdésenkénti költség 0,00137, illetve 0,00148 USD volt, a medián késleltetés 7,1, illetve 7,5 s; a medián késleltetés a kérdésenkénti válaszidő tipikus értéke (a kérdések felére ennél gyorsabban, felére lassabban érkezett válasz).

Új kérdéstípusok. Benedek a rendszert olyan kérdéstípusokon is megmérte, amelyeket a hangolás során nem használt (100 kérdés). Ezeken a hangolt kaszkád pontossága kb. 100 lapon 0,770, kb. 1000 lapon 0,740 volt. Az elmaradás szinte teljesen a lefelé eső elzáró szelepeket kereső kérdésekből jön: ezek közül a 20-ból mindkét méretnél csak 2 volt helyes.

Ezeket a számokat jelen munka nem méri újra; a 6. fejezetben a gráf nélküli módszer skálázódásának hátteréül szolgálnak.

2.3 Kérdés-válaszolás műszaki dokumentumokon

Gyártási dokumentumokon a Document GraphRAG a dokumentum saját szerkezetéből és egy könnyű, automatikusan épített példánygráfból épít tudásgráfot[24]; a szerzők elismerik, hogy szakterületen az automatikus tudásgráf-építés pontatlan, ezért ma is főleg kézi munka. Ipari tudásbázison a Golden-Retriever a visszakeresés előtt egy reflexiós lépésben azonosítja és tisztázza a szakzsargont és a rövidítéseket[13]. A műszaki dokumentumokba ágyazott táblázatokat és szerkezetet a visszakeresésben külön kezelő, strukturált adatokat figyelembe vevő RAG-irány[43] azt jelzi, hogy az adatlapi táblázatok a sima szövegdarabolástól eltérő kezelést igényelnek. Mérnöki rajzokat és kézikönyveket feldolgozó látás-nyelvi modellek egy friss benchmarkon megtalálják a tételeket, de gyakran pontatlanul írják le őket, és a szövegátfedésen alapuló metrikák alulmérik a képességüket; a szerzők ezért

keresztbírálós LLM-értékelést használnak[26]. Ezért a szabad szöveges válaszokat mi is bírálóval pontozzuk (3. fejezet).

Szabó Zsombor egyéni munkája (plantdocs) egyetlen eszköz adatlapjából épít ISO 15926 / QUDT szerinti tudásgráfot, amelyben a feltételek és a kereszthivatkozások külön elemek maradnak. Két lekérdezési módszert hasonlít össze: A1 (RDF + SPARQL) és A2 (Neo4j + Cypher, vektoros kereséssel). A plantdocs végső mérésén (a Rosemount 3051 adatlap 200 befagyasztott kérdésén, vak bírálóval) az A1 141, az A2 172 kérdésre adott helyes választ. Ebben a munkában az A2 az adatlapoldal alapja.

2.4 GraphRAG és visszakeresési stratégiák

Mikor segít a gráf?

A gráfalapú RAG nem mindig jobb a szövegalapúnál. Egy benchmark szöveges korpuszon méretfüggő keresztezést mutatott ki a sima RAG és egy gráfalapú módszer között: kis korpuszon a sima RAG nyer, és a gráf előnye a korpusz méretével jelenik meg[46]. A ChatP&ID egylapos eredménye és Benedek üzemi méretű mérései ugyanezt a mintát mutatják a rajzokon; kis üzemen (például a mi négylapos prototípus-üzemünkön) ezért a gráf nélküli módszer erős ellenfél.

Ha több visszakeresési stratégia áll rendelkezésre, kérdésenként lehet közöttük választani. A GraphRAG-Router megerősítéses tanulással tanított útválasztója kérdésenként választ GraphRAG-módszer és nyelvi modell között, és ezzel csökkenti a nagy modell hívásainak számát[10]. A mi M2 routerünk ennek egyszerű, tanítás nélküli változata: egy nyelvi modell dönti el lépésenként, melyik meglévő rendszert hívja.

Gráfeszközöket hívó ágensek

A tudásgráf-kérdés-válaszolás bevett mintája az ágens, amely lépésenként eszközöket hív a gráfon, és az eredményekből állítja össze a választ. A KBQA-Agent hét tudásbázis-eszközt ad (relációk, szomszédok, attribútumok, legnagyobb és legkisebb érték, metszet, darabszám); minden lépés eredménye nevet kap, így egy későbbi lépés név szerint visszautalhat rá, és azon dolgozhat tovább (például előbb a szomszédokat kéri le, majd ezek közül választja ki a legnagyobb értékűt)[22]. Hasonló mintát követ a KG-Agent[23] és a Graph-CoT[14]. A skála másik végén a szabad Cypher áll: a Neo4j MCP-szervere mindössze sémalekérdező, olvasó és író Cypher-eszközt ad, a lekérdezést a modell írja[29].

Benedek ágense a KBQA-Agent családjába tartozik: hat általános, egymással komponálható üzemi primitívet hív, az eredményeket hivatkozással adja tovább; a mi `datasheet_fact` primitívünk ezt a készletet bővíti (5. fejezet).

Kaszkád és önjelzés

A kaszkád (lépcsős módszer) a kérdést előbb a legolcsóbb módszernek adja, és csak akkor viszi tovább, ha az olcsó válasz gyanús. A gyanú forrása önjelzés: olyan jel, amelyet a lépcső maga ad a válaszáról, a helyes válasz ismerete nélkül, például a hibát dobó vagy üres lekérdezés. Az ilyen szabály nem illeszt paramétert a mért adatra, ezért kevésbé hajlamos a

túlillesztésre, mint a kérdéstípus szerinti útválasztás. Benedek kaszkádja is ilyen szabállyal dönt; az 5. fejezetben részletezzük.

2.5 Összevetés

Az alábbi táblázat azt foglalja össze, hogy a kapcsolódó munkák a probléma mely részeit fedik le; a kiértékelés méretét a forrás szerint adjuk meg.

1. táblázat: A kapcsolódó munkák összevetése

Munka	Rajz	Adatlap	Több lap	Gráf	Kiértékelés mérete
ChatP&ID[4]	igen (DEXPI)	nem	nem, egy lap	Neo4j, tulajdonsággráf	19 kérdés
Talking like P&IDs[3]	igen (DEXPI)	nem	nem	sűrített gráf a promptban	3 kérdés, egy folyamatábra
PIDQA[15]	igen (szintetikus, raszteres)	nem	nem, kérdésenként egy lap	Neo4j, tulajdonsággráf, Text2Cypher	64 000 kérdés, 500 lap
Gadekar és társai[11]	igen (raszteres)	nem	nem	visszanyert folyamatgráf, 7 operátor	3000 kérdés, szintetikus lapok
pid-graph-agent[35]	igen (DEXPI)	nem	nem, kérdésenként egy fájl	gráfeszközös ágens	15 + 37 saját kérdés
Everllence-esettanulmány[25]	igen	nem	igen	Neo4j, ágens	nem közöl
Lopomo és társai[27]	igen (126 rajz)	nem	nincs adat	automatikusan épített tudásgráf	gráfépítés, nem kérdés-válaszolás
Document GraphRAG[24]	nem	gyártási dokumentumok	–	dokumentumszerkezet és példánygráf	nincs adat
Enginuity[26]	mérnöki rajzok	kézikönyvek, alkatrésztáblák	–	nincs	6940 oldal
plantgraph (Kiszely Benedek)	igen (szintetikus)	nem	igen, kb. 100 és 1000 lap	Neo4j, üzemi réteg	460 kérdés a lezárt teszten
plantdocs (Szabó Zsombor)	nem	igen, egy eszköz	–	ISO 15926 / QUDT, RDF és Neo4j	200 kérdés
Jelen munka	igen (30 és 50 lapos fejlesztési, 100 és 1000 lapos tesztüzem)	igen, 7 adatlap	igen	összevont Neo4j-gráf	üzemenként 60 kérdés + 12 (e) kérdés

A rajzoldali munkák szinte mind egylaposak, és egyikük sem kapcsolja be az eszközök adatlapjait; az egyetlen többlapos ipari példa kiértékelést nem közöl. A dokumentumoldali munkák pedig nem használják a rajz topológiáját.

2.6 A hiányzó lépés

A két egyéni munka ugyanannak az üzemi dokumentációnak két oldalát dolgozza fel. A rajzon látszik, hogy mi mivel áll kapcsolatban, az adatlapon, hogy egy eszköz mire képes. Egy mérnöki kérdés gyakran mindkettőt igényli („mekkora a maximális nyomás azon a

szelepen, amely a P-101 nyomóoldalát elzárja?”). Az általunk áttekintett irodalomban nem találtunk olyan munkát, amely a rajzi gráfot és az eszközönkénti adatlapgráfokat egy lekérdezhető modellben kötné össze, és ezt mérné. Ez a dolgozat tárgya.

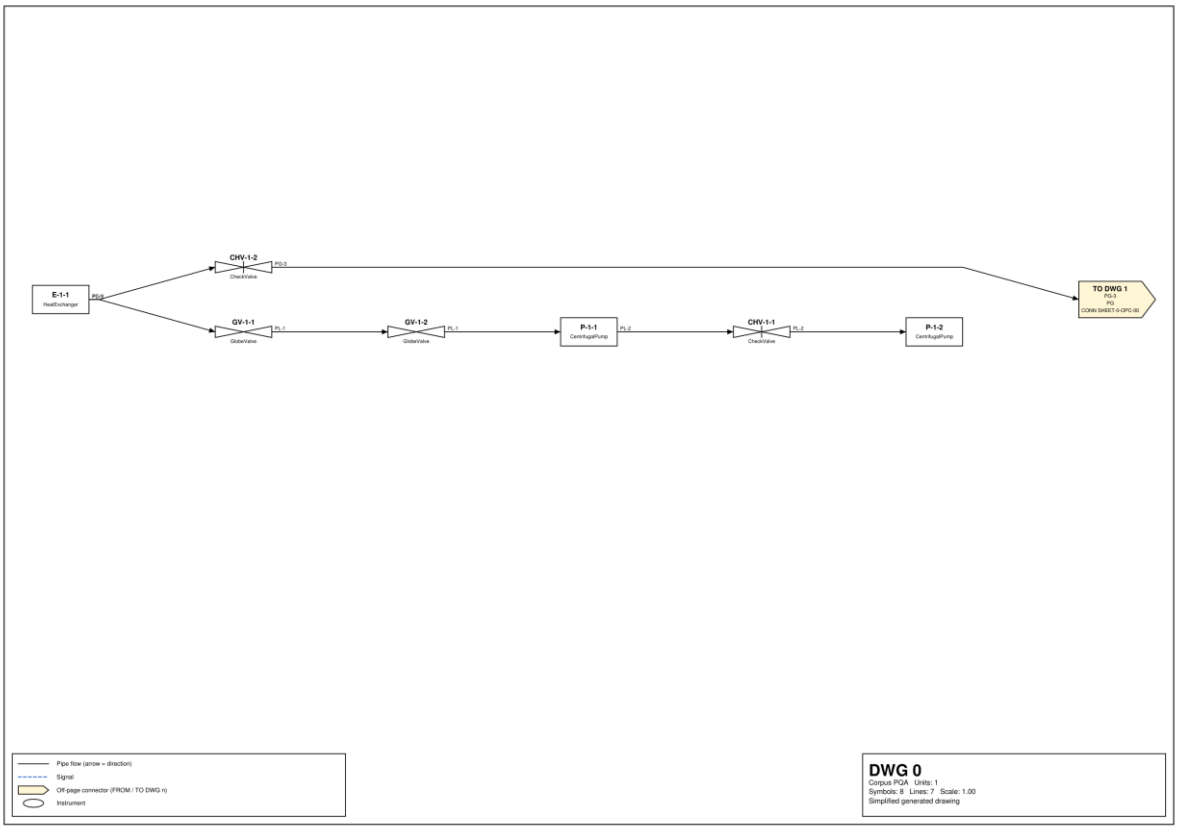
3. Adatok és feladat

Ebben a fejezetben bemutatjuk a mérés tárgyát: a példaüzemek rajzát, a tagelt elemeikhez tartozó gyártói adatlapokat és az üzemenként 60 kérdésből álló kérdéssort. Az első prototípus egy kis, négylapos üzemen készült. A közös rendszert ezután két fejlesztési üzemen (30 és 50 lap) hangoltuk, a hangolt és befagyasztott rendszert pedig két tesztüzemen (100 és 1000 lap) mértük meg, amelyeket a hangolás során nem használtunk.

3.1 A kis példaüzem (az első prototípus)

Az első prototípushoz egy közepesen bonyolult, többlapos P&ID-t generáltunk Kiszely Benedek generátorával (plantgraph), annak egyszerűbb, első változatával. A generátor véletlenszerűen dönt; a seed a véletlenszám-generátorának kezdőértéke, és ugyanaz a seed mindig ugyanazt az üzemet adja, így az üzem pontosan újra előállítható. A valósághűbb, későbbi változatot azért nem használhattuk, mert csak az első változatban lehet a generált berendezésosztályokat (a berendezéssúlyokkal) azokra szűkíteni, amelyekhez van gyártói adatlapunk, és ez az adatlapok hozzárendeléséhez szükséges; a későbbi változat olyan berendezésosztályokat is generál, amelyekhez nincs adatlapunk.

A két változat a következőben különbözik. Az első változat hat berendezésosztállyal (centrifugálszivattyú, kompresszor, tartály, nyomástartó edény, oszlop, hőcserélő) és háromféle szeleppel (globe, golyós, visszacsapó) dolgozik; a berendezéseket véletlen faszerkezetben köti össze, és műszer csak a szabályozókörökben van. A későbbi változat valódi folyamategységek mintáiból építi az üzemet (például oszloprendszer, kompresszorfokozat, hevítő–reaktor–szeparátor sor, tartálypark, hőcserélősor, szűrőegység), a berendezések köré a szokásos kiegészítő elrendezéseket rajzolja (szivattyú szívó- és nyomóoldala, szabályozószelep-állomás, biztonsági szelep, leürítők, légtelenítők), sokkal több berendezés-, csővezeték- és műszerfajtát használ, és az elemek között jelöletlen idomok is állnak. Ez valószerűbb rajzot ad, de a berendezésosztályok nem szűkíthetők az adatlappal rendelkezőkre. A generátor pyDEXPI-modellt épít, a lapokra bontó a vágott vezetékekből lapok közötti csatlakozópárokat (off-page connector) képez, és maga a bontás a megoldókulcs. A generátort nem módosítottuk, két ponton szűkítettünk. Először a berendezéssúlyokkal csak olyan berendezésosztályokat engedtünk meg, amelyekhez van adatlap (centrifugálszivattyú, hőcserélő). Másodszor a szabályozóköröket szűrtük. A generátor szabályozóköröket is tesz az üzembe, és mindegyik kör egy változót szabályoz: áramlást, szintet, nyomást vagy hőmérsékletet. A kör távadójához adatlap kell, a gyűjteményünk egyetlen távadó-adatlapja (Rosemount 3051) viszont nyomás- és nyomáskülönbség-távadóé, amely a nyomás és az áramlás mérését lefedi, a szintét és a hőmérsékletét nem. A kis prototípus-üzemhez ezért csak olyan üzemet fogadtunk el, amelynek minden köre áramlás- vagy nyomásszabályozó kör. A seedeket sorban (1, 2, 3, …) kipróbáltuk, és az első olyan üzemet fogadtuk el, amely ezen felül 20–40 tagelt elemet és 2–4 lapot tartalmaz, és amelyben minden adatlaptípus szerepel (ez a 6-os seed volt). Az üzem 26 tagelt elemből és 4 lapból áll, 4 lapközi csatlakozópárral; a Neo4j-ben tárolt üzemi rétege 36 csúcs és 116 kapcsolat. A visszaépítési ellenőrzés (a lapokból feloldott üzem azonos-e a generálttal) hibátlan volt.



1. ábra: A kis példaüzem egyik lapja. A jobb szélen a lapközi csatlakozó.

3.2 A fejlesztési és a tesztüzemek

A kis üzem négy lapja egy valódi üzemnél nagyságrendekkel kisebb, ezért a közös rendszert négy nagyobb üzemen hangoltuk és mértük. A két fejlesztési üzem pontosan 30 és 50 lapos; ezeken hangoltuk a módszereket. A két tesztüzem pontosan 100 és 1000 lapos; ezeket a hangolás során nem használtuk, és a hangolt, befagyasztott módszereket ezeken mértük meg. Mind a négyet ugyanazzal a generátorral, ugyanazokkal a berendezéssúlyokkal és ugyanazzal a hét adatlappal generáltuk; az egységszámot (17, 28, 57, illetve 600) úgy választottuk, hogy a lapszám pontosan elérhető legyen, és az elfogadási feltétel a pontos lapszám volt. A nagyobb üzemeknél nem követeltük meg, hogy minden kör áramlás- vagy nyomásszabályozó legyen, mert ennyi körnél szinte minden üzemben van szint- és hőmérséklet-szabályozó kör is, így egyetlen seed sem felelt volna meg. Ehelyett a szint- és hőmérséklet-távadók egyszerűen nem kapnak adatlapot, ahogy egy valódi üzemben sincs minden elem adatlapja kéznél.

2. táblázat: A fejlesztési és a tesztüzemek

Szerep	Lap	Egység	Tagelt elem	Adatlappal	Szabályozófunkció adatlap nélkül	Szint- vagy hőmérséklet-távadó adatlap nélkül
fejlesztési	30	17	206	174	19	13
fejlesztési	50	28	378	324	40	14
teszt	100	57	772	644	87	41

Szerep	Lap	Egység	Tagelt elem	Adatlappal	Szabályozófunkció adatlap nélkül	Szint- vagy hőmérséklet-távadó adatlap nélkül
teszt	1000	600	7854	6641	821	392

Egy adatlap így több tag-et is leír: a 30 lapos üzemen például 53 szivattyú ugyanazt a Gorman-Rupp adatlapot kapja, ahogy egy valódi üzemben is ugyanaz a típus több helyen beépül. A 30, 100 és 1000 lapos üzem lapszámban 7,5-szer, 25-ször, illetve 250-szer, a tagelt elemek számában körülbelül 8-szor, 30-szor, illetve 300-szor nagyobb a kis üzemnél. A generátor szabályokra épül (egységek, berendezések, csővezetékek, ISA-szerű tag-ek, szabályozókörök, lapközi csatlakozók), de az üzemek nem létező üzemek rajzai: a kapcsolások véletlenszerűek, a technológia nincs méretezve. A rajz tehát véletlenszerű, de belsőleg következetes és teljesen ismert: a generátor minden elemet, kapcsolatot és szabályozókört pontosan ismer. A kérdéseket és a referenciaválaszokat program állítja elő éppen ebből az ismert szerkezetből (az adatlapi részt az adatlapokból), ezért minden kérdésnek egyértelmű, ellenőrizhető válasza van; ehhez nem szükséges, hogy a technológia valószerű legyen.

3.3 Adatlapok

A tagelt elemek osztálya és szerepe határozza meg az adatlapot (lásd az alábbi táblázatot). A hozzárendelést program állítja elő a generátor kimenetéből, nem kézzel. A szabályozókör által működtetett globe szelep a Fisher ET szabályozószelep, a kör nélküli globe szelep a GF 165 mágnesszelep (kétutas, globe kialakítású nyitó-záró szelep). A két szabályozófunkcióhoz (FIC, PIC) nem rendeltünk adatlapot, mert ezek a folyamatirányító rendszer funkciói, gyártói adatlapjuk nincs.

3. táblázat: Elemosztályok és adatlapok a kis példaüzemben

DEXPI-osztály (szerep)	Tag-ek	Adatlap	Oldal
CentrifugalPump	P-1-1 … P-2-3 (8 db)	Gorman-Rupp VGH (0700286)	6
HeatExchanger	E-1-1, E-2-1, E-2-2, E-2-3	Alfa Laval M3	2
GlobeValve (szabályozókörben)	FV-2-2, PV-2-1	Fisher ET (D100022X012)	36
GlobeValve (kör nélkül)	GV-1-1, GV-1-2, GV-2-2	GF Type 165 mágnesszelep	6
BallValve	BV-2-1	Flowserve Worcester F51/F52	12
CheckValve	CHV-1-1 … CHV-2-2 (4 db)	Swagelok MS-01-176	16
ProcessSignalGeneratingFunction	FT-2-2, PT-2-1	Rosemount 3051 (00813-0100-4001)	114

Az adatlapokból a plantdocs tízlépéses feldolgozó folyamata épít ISO 15926 / QUDT szerinti tudásgráfot (a lépéseket a 4. fejezet és a Függelék D írja le). A Fisher, a GF 165 és a Gorman-

Rupp adatlapjának gráfja Szabó Zsombor egyéni munkájából származik; a másik négy adatlapot (Rosemount 3051, Alfa Laval M3, Worcester F51/F52, Swagelok MS-01-176) mi dolgoztuk fel ugyanezzel a folyamattal. Az így kapott gráfok mérete 91 202 (Rosemount 3051), 1102 (Alfa Laval M3), 16 281 (Worcester F51/F52) és 66 747 (Swagelok MS-01-176) triplet.

3.4 A kérdéssor

A kis üzem kérdéssora 60 kérdésből áll, négy családban (lásd a második táblázatot ebben a fejezetben). Családonként 5 kérdés a fejlesztési (dev) halmazba, 10 a teszthalmazba került; a teszthalmazt az első prototípus fejlesztése előtt befagyasztottuk (ellenőrzőösszeggel rögzítve), és csak a prototípus végső mérésénél futtattuk. A két fejlesztési és a két tesztüzem kérdéssorát ugyanaz a program, ugyanazzal a seeddel állította elő. A fejlesztési üzemek kérdéseit a hangoláshoz használtuk; a tesztüzemek kérdéseit a hangolás alatt nem futtattuk, ott mind a 60 kérdés tesztkérdés. Minden üzemhez tartozik 12 további, több adatlapot összevető kérdés is (az (e) család).

4. táblázat: A kérdéscsaládok

Család	Mit kérdez	Referenciaválasz forrása	Dev / teszt
(a) csak rajz	kapcsolat, áramlási út, felső források, műszerek, típus, egység, lapok	Benedek kérdésgenerátora a generált üzem pontos leírásából	5 / 10
(b) csak adatlap, tag-en át	„Regarding P-1-3 on the P&ID: …”	Szabó Zsombor egyéni munkájának kérdés–válasz párjai	5 / 10
(c) összetett	egy topológiai lépés és egy adatlapi tény	programmal összerakva az (a) és a (b) család referenciáiból	5 / 10
(d) megválaszolhatatlan	nem létező tag, a dokumentumban nem szereplő tény	Benedek generátora, plantdocs „not in document” párok	5 / 10

Az összetett kérdések topológiai lépését a generált üzem pontos leírásából program számolja: csak egyértelmű lépés marad (pl. „a P-1-1 előtt közvetlenül álló elem”, „az E-2-1-et mérő távadó”, „a PIC-2-1 kör által működtetett szelep”), és csak olyan, amelynek cél-eleméhez van referenciaválasszal ellátott adatlapi kérdés. A kérdés szövege a lépés és az eredeti adatlapi kérdés összefűzése.

3.5 Pontozás

Ahol a válasz programmal ellenőrizhető, programmal pontozunk. Az (a) család és a rajzos megválaszolhatatlan kérdések válaszát egy program a válasz típusa szerint veti össze az ismert referenciaválasszal: egy tag-halmaznak halmazként pontosan egyeznie kell, egy útnak ugyanazokat az elemeket kell tartalmaznia ugyanabban a sorrendben, egy számnak egyenlőnek kell lennie, egy igen/nem válasznak egyeznie kell, a megválaszolhatatlan kérdésekre pedig a tartózkodás a helyes válasz (vagyis annak kimondása, hogy a dokumentumok ezt nem tartalmazzák). A szabad szöveges kérdéseknél programmal helyes

a válasz, ha a referenciaválasz minden kötelező eleme számot tartalmaz, és minden szám a mértékegységével együtt megjelenik a válaszban; programmal rossz a megválaszolható kérdésre adott „nincs benne”, programmal helyes a megválaszolhatatlanra adott „nincs benne”.

A többi választ egy vak bíróként működő nyelvi modell pontozza, rögzített pontozási utasítással. A bíró csak a kérdést, a referenciaválaszt és a jelölt választ látja, kevert sorrendben, és nem tudja, melyik módszer és melyik modell adta a választ. Négy ítélet lehetséges: helyes, részben helyes, rossz és hallucinált (a válasz olyan tényt állít, amely nincs benne a dokumentumokban). Pontosságnak a helyes válaszok arányát tekintjük.

4. Az összevont tudásgráf

A két egyéni munka két külön gráfot épít: a rajzból egy üzemi réteget (Benedek), az adatlapokból eszközönként egy-egy adatlapgráfot (plantdocs). Az összevonás célja, hogy egyetlen adatbázisban egyetlen lekérdezés mindkettőt elérje. Ebben a fejezetben előbb röviden összefoglaljuk a két forrásfolyamatot és az általuk épített réteget, utána a két réteg közötti kapcsolatot és a betöltés mérését. A két folyamat kódját nem módosítottuk; részletes leírásuk, amely a két egyéni munka terveiből és kódjából származik, a Függelék D-ben található.

4.1 A rajzoldali réteg

Benedek rendszerében a generátor egy teljes üzemet épít, a lapokra bontó (splitter) lapokra vágja, a feloldó (resolver) a lapokból megoldókulcs nélkül visszaépíti az üzemet, végül a betöltő Neo4j-be tölti (részletesen: Függelék D). A generátor minden véletlen döntést egyetlen seedelt véletlenszám-generátoron át hoz meg, ezért ugyanaz a seed ugyanazt az üzemet adja; a kimenet valódi pyDEXPI-modell elágazó topológiával, a berendezésekhez tartozó szabályozókörökkel és a szokásos tagbetűkkel. A példaüzemek a generátor egyszerűbb, első változatával készültek, mert a valósághűbb, későbbi változat berendezésosztályai nem szűkíthetők az adatlappal rendelkező osztályokra. A bontó minden elvágott élre lapközi csatlakozópárt illeszt, a megoldókulcsot pedig külön fájlba írja, amely a vizsgált adatbázisba nem kerül be. A feloldó csak a lapokat látja: a csatlakozókat a feliratuk kölcsönös hivatkozása alapján párosítja, a több lapon felrajzolt elemeket a (tag, osztály) kulcs szerint vonja össze, és nem találgat. Mivel a bontást a rendszer maga végzi, a feloldó eredménye a megoldókulcshoz mérhető. A példaüzem 4 lapja között 4 lapközi csatlakozópár van; a rajzexportban mind a 8 lapközi jel összekötött, a visszaépítési ellenőrzés pedig hibátlan volt.

Az üzemi réteg és a vonalnézet

Benedek döntése szerint az adatbázisban két réteg lehet: a rajz szerinti előfordulásréteg (rajzolt szimbólumok, lapközi csonkok, azonossági élek) a nyomon követésre, és az összevont üzemi réteg, a tulajdonképpeni tudásgráf, amely kizárólag a feloldó kimenetéből épül. Az üzemi rétegben egy laphatár átlépése egyetlen él, az előfordulásrétegben több lépéses minta.

A generált üzemet Benedek betöltője tölti a Neo4j-adatbázisba. Minden rajzi elem pontosan egy csúcs lesz, akkor is, ha több lapon is szerepel. A csúcs több szinten is hordozza az elem típusát: egy centrifugálszivattyú egyben szivattyú és berendezés is, így egy lekérdezés bármelyik szinten rákereshet. Az elemek közötti csöveket az áramlás irányába mutató kapcsolatok írják le (`send_to`); ha egy ilyen kapcsolat átlépi a laphatárt, ezt a kapcsolat jelölése is rögzíti. A jelvezetékeknek, a mérésnek (melyik távadó melyik elemet méri) és a beavatkozásnak (melyik szabályozó melyik szelepet működteti) külön kapcsolattípusa van. Végül minden elem kapcsolatban áll azzal a lappal, amelyen szerepel, és azzal az egységgel, amelyhez tartozik.

Erre épül a vonalnézet. Ez egy rövidítő kapcsolat (`line_to`), amely két berendezést vagy szelepet közvetlenül köt össze, és átugorja a köztük lévő kisebb csőidomokat. Így az olyan kérdés, mint „mi a szivattyú utáni következő szelep?”, egyetlen lépésben megválaszolható; az átugrott elemeket a rövidítő kapcsolat megjegyzi. A csövenkénti kapcsolatok is megmaradnak, így a részletes nézet sem vész el. A vonalnézetet Benedek a generátor valósághűbb változatának üzemeihez vezette be, ahol sok jelöletlen idom áll az elemek között; a mi üzemeinkben ilyen idom nincs, de a kaszkád promptjai és lekérdezési szabályai erre a nézetre készültek, ezért ezt használjuk. A kis prototípus-üzem üzemi rétege 36 csúcsból és 116 kapcsolatból áll.

4.2 Az adatlapoldali réteg

A plantdocs egy gyártói adatlap PDF-jéből tíz számozott lépésben jut el az ontológia szerinti RDF-gráfig:

1. elrendezés-elemzés az Azure Document Intelligence szolgáltatással (bekezdések, táblázatok, ábrák);
2. rekordok: egy nyelvi modell a táblázatokból paraméter–érték–egység rekordokat ír, a táblázatbeli útvonallal;
3. ábrák: kivágás, osztályozás és a rajzokon lévő jelek feloldása;
4. diagramok leolvasása;
5. ellenőrzés: minden rekord számának megkeresése a PDF szövegében;
6. célformátum: a rekordokból ontológiafüggetlen tények és feltételek;
7. lábjegyzetek: az ábrákhoz tartozó lábjegyzetek bekötése;
8. forrásleírások: minden forrásról (táblázatról, ábráról, szövegrészről) rövid leírás;
9. szókészlet-illesztés: a tulajdonságokhoz és feltételekhez szabványos fogalmak rendelése;
10. RDF: a gráf előállítása.

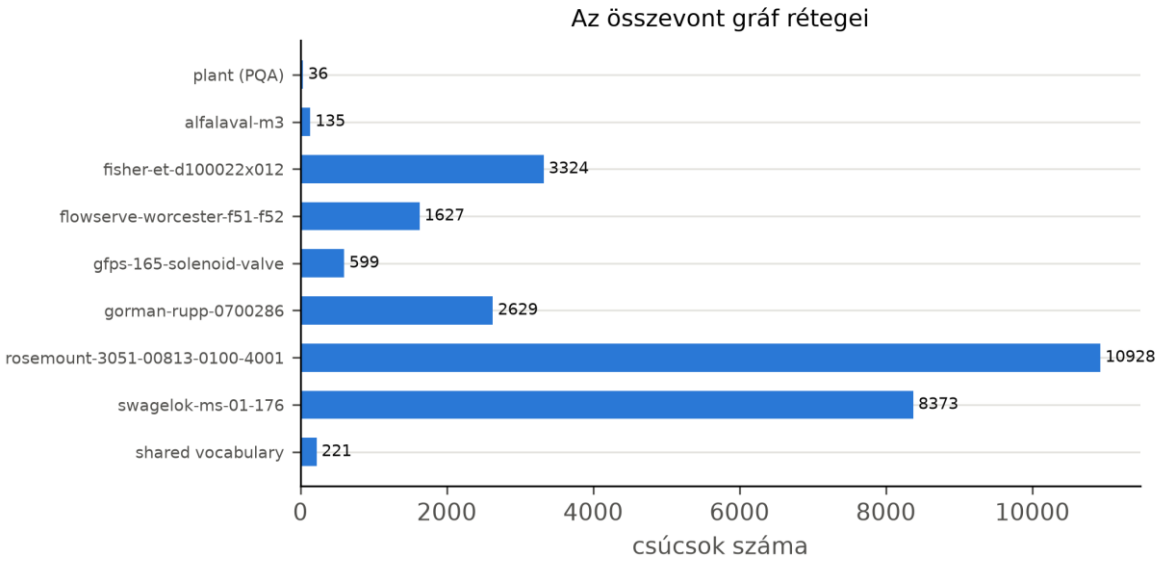
A lépések felváltva programok és nyelvi modellek; minden nyelvimodell-lépés kimenete csak akkor mehet tovább, ha átmegy egy programmal ellenőrzött formai és tartalmi feltételrendszeren. Az általunk feldolgozott négy adatlapon minden nyelvimodell-kimenet elsőre átment ezen az ellenőrzésen.

A célformátumban egy rekord egy tény, nem egy nyomtatott cella; a feltételek elsőrendű elemek, a nyomtatott forma megmarad, és minden visszakövethető a PDF-oldalig. Az RDF-modell a plantdocs ontológiai döntése szerint az ISO 15926 vonalat követi: az IDO a felső ontológia, a POSC Caesar könyvtára az eszköz- és referenciaszótár, a mértékegység QUDT, saját, kézzel írt szókészlet pedig nem lehet. Egy mennyiségi tény mennyiségi adatpont (Quantity Datum), a feltételek együttese egy esetcsúcs (Case), amelyre a tény és minden egyes feltétel mutat, a forrás szakaszcsúcs (Section Datum), a lábjegyzet külön adatpont (részletesen: Függelék D).

Az adatlapréteg a Neo4j-ben

Egy adatlapgráf a plantdocs RDF-kimenete, amelyet a neosemantics (n10s) bővítmény tölt be: minden csúcs `:Resource {uri}`, az RDF-típusok további címkék (például a mennyiségi adatpont címkéje `plm__PCA_100029511`), a szabványos azonosítók rövidített előtagos

neveket kapnak. Az eszköz gyökércsúcsának URI-ja `urn:plantdocs:<doc>:device`. Az adatpontok az eszközre vagy egy alkatrészére mutatnak, a feltételeket esetcsúcs, a lábjegyzeteket külön adatpont, a forrást (táblázat, ábra, szakasz) forrásszakasz-csúcs hordozza oldalszámmal. Ez a szerkezet őrzi meg, hogy egy érték milyen feltétel mellett és melyik oldalon érvényes. A hét eszközgráf együtt 27 615 csúcsból és 103 366 kapcsolatból áll; a közös szókészlet-csúcsok (egységek, osztályok, szabványos attribútumok) száma 221.



2. ábra: Az összevont gráf csúcsai rétegenként és adatlaponként.

4.3 Az összekötés

A két réteget egy kapcsolat köti össze: `(:PlantItem {tag})-[:DESCRIBED_BY]->(eszköz)`. A kapcsolatot a program által előállított tag → adatlap tábla alapján töltjük be; a 26 tagelt elemből 24-nek van adatlapja, ennyi `DESCRIBED_BY` él jön létre. Több tag mutathat ugyanarra az eszközgráfra: a nyolc szivattyú ugyanazt a Gorman-Rupp adatlapot kapja, mert ugyanaz a típus. A névütközést az URI-k dokumentumonkénti előtagja zárja ki; emellett minden adatlapcsúcs `document_id` tulajdonságot kap, amellyel a lekérdezés egy dokumentumra szűkíthető. A közös szókészlet-csúcsok (például egy mértékegység) az eszközök között közösek.

Egy összetett kérdés így egyetlen bejárásként fogalmazható meg: a rajzi rétegben megkeressük a kérdéses elemet (például a P-1-1 utáni első szelepet a `line_to` mentén), a `DESCRIBED_BY` kapcsolaton átlépünk az adatlapjára, és ott a megfelelő adatpontot olvassuk ki a feltételével és az oldalszámával. Az alábbi minta az összevont gráf egy részletét mutatja Cypher-jelöléssel: egy szivattyút, a rajzi szomszédját, az adatlapja gyökércsúcsát és egy adatpontot a környezetével.

```
// rajzi réteg (Benedek, vonalnézet)
(p:PlantItem:CentrifugalPump {tag: 'P-1-3'})-[:line_to]->(v:PlantItem:Valve)
(p)-[:is_drawn_on]->(:Sheet)

// az összekötés
(p)-[:DESCRIBED_BY]->(dev:Resource:lis__PhysicalObject)

// adatlapi réteg (plantdocs, egy adatpont a környezetével)
```

```
(d:plm__PCA_100029511)-[:plm__PCA_100029516]->(dev)                // alany: az eszköz
vagy egy alkatrésze
(d)-[:plm__PCA_100029520]->(c:plm__PCA_100029524)                  // eset: a feltételek
együttese
(cond:plm__PCA_100029508)-[:plm__PCA_100029520]->(c)               // egy feltétel
ugyanabban az esetben
(d)-[:plm__PCA_100029525]->(s:plm__PCA_100029513)                  // forrásszakasz,
oldalszámmal
(d)-[:plm__PCA_100029514]->(f:plm__PCA_100029505)                  // lábjegyzet
(d)-[:lis__datumUOM]->(u:Resource)                                 // mértékegység, QUDT-
megfeleltetéssel
```

Az összekötés módját több alternatívával szemben választottuk. A legegyszerűbb az lett volna, ha az adatlapi tényeket tulajdonságként a rajzi csúcsra másoljuk; ez azonban éppen a feltételeket, a lábjegyzeteket és az oldalszámot veszítené el, amelyekért a plantdocs gráfja készült. Két külön adatbázis között egy lekérdezés nem járhat át (és a helyi Neo4j Community kiadás csak egy felhasználói adatbázist enged); az egész gráf RDF-ben pedig Benedek Neo4j-re írt kaszkádjának módosítását igényelte volna. A tagalapú él a legkisebb beavatkozás: egyik réteg szerkezete sem változik, és a kapcsolat a gyakorlatot követi, ahol a tag köti össze a rajzot a többi dokumentummal. Ára, hogy a tag → adatlap táblát elő kell állítani; nálunk ezt program végzi a generátor kimenetéből (3. fejezet), valódi üzemben ez az eszközlista vagy az átadási adatbázis dolga volna.

4.4 Betöltés

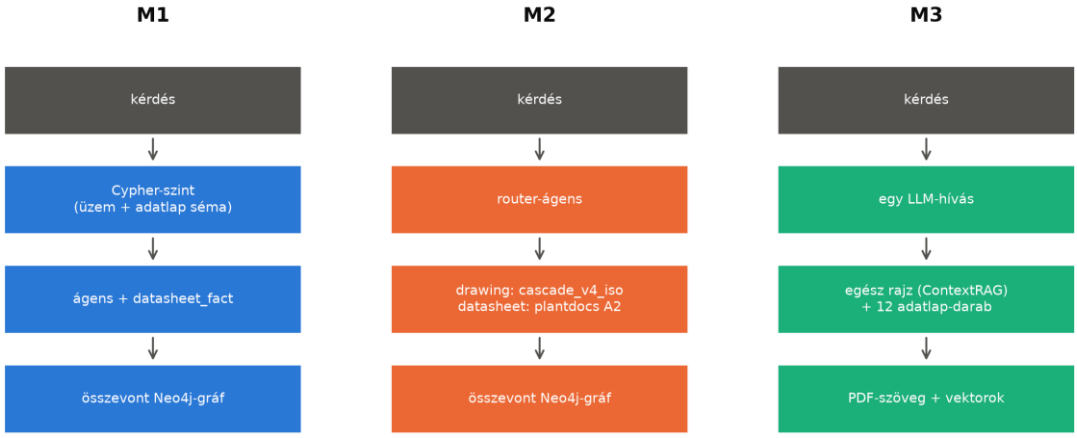
A betöltés egy paranccsal újrafuttatható. Sorrendje: az adatbázis törlése és a neosemantics beállítása, az eszközgráfok betöltése és névtérbe sorolása, az üzemi réteg betöltése Benedek betöltőjével (amely csak a saját korpuszát törli), a `DESCRIBED_BY` élek, végül a plantdocs A2 vektoros kereséséhez szükséges beágyazások. A teljes betöltés 243,2 s; ennek nagy része (222,4 s) a 14 489 beágyazás előállítása, tárolása és indexelése, az üzemi réteg 3,6 s alatt töltődik be. A helyi Neo4j Community kiadás csak egy felhasználói adatbázist enged, ezért a betöltő minden futáskor az egész adatbázist újraépíti.

Két illesztési pontra külön figyelni kellett. Benedek tárellenőrzése összeveti az adatbázis kapcsolatszámait a betöltési tervvel, és eltérésnél nem indítja el a kaszkádot; ezért a kaszkád egy leszármazott nézeten át olvas, amely a `DESCRIBED_BY` kapcsolatot nem számolja a rajzi korpuszhoz. A másik pont a plantdocs: az alkalmazása dokumentumonként törli és újratölti az adatbázist, ami az összevont gráfot tönkretenné. Ezért az A2-t nem az alkalmazáson át hívjuk, hanem közvetlenül a lekérdező függvényét, egy dokumentumra szűkítő lekérdezés-futtatóval (5. fejezet).

5. Kérdés-megválaszolási módszerek

Három módszert hasonlítunk össze. Mindhárom a két meglévő rendszerre épül, azokat nem módosítja: a kódjukat importáljuk, és csak a hiányzó kapcsolatot írjuk meg. A három módszer ugyanazt a kérdést kapja (szöveg és elvárt válaszalak), és ugyanabban az alakban válaszol: egy JSON-objektum a válasszal és egy jelzővel arra, hogy a dokumentumok nem tartalmazzák a választ. Mindhárom módszert mindkét kérdés-válaszoló modellel mérjük: a `z-ai/glm-5.3-flash` és az `openai/gpt-6-luna` modellel, Benedek rögzített paramétereivel (pinjeivel).

A válaszalak Benedek kérdéstípusait követi: szabad szöveg (`FREE_TEXT`), egy tag vagy taghalmaz (`TAG`, `TAG_SET`), rendezett út (`TAG_PATH`), igen/nem (`BOOLEAN`), darabszám (`COUNT`), egység (`UNIT_ID`) és osztálynév (`CLASS_NAME`). A válaszalakot leíró mondatot mindhárom módszer Benedek kódjából veszi, így a válaszíró modell mindhárom esetben ugyanazt az utasítást látja. A fejezetben előbb a közös építőelemeket, majd a három módszert és a tervezési döntéseiket mutatjuk be, végül megfogalmazzuk, mit várunk az egyes módszerektől.



3. ábra: A három módszer felépítése. Bal: M1 (egy összevont gráf); közép: M2 (router a két rendszer fölött); jobb: M3 (gráf nélküli baseline).

5.1 Közös építőelemek

A rajzoldali kaszkád

Benedek legutóbb mért kaszkádja két szintből áll. A kaszkádot Benedek a generátor valósághűbb, későbbi változatának üzemein hangolta egy külön rendszerként, a pontosságot helyezve előtérbe. A hangolás többek között a jelöletlen idomokat (ehhez készült a 4. fejezetben leírt vonalnézet, mindkét szintre ugyanazzal a szabállyal), a Cypher-lekérdezések szintaktikai hibáit és a korlátlan útkereséstől lejáró lekérdezéseket kezelte. A kaszkádnak két változata volt (az egyik az alább leírt címke szerinti indítással); a használtat egy előre rögzített szabály választotta: a címke szerinti indítású változat akkor lép a másik helyébe, ha a hangolókészleten legalább olyan pontos.

Első szint: egy Cypher-lekérdezés. Egy LLM egyetlen Cypher-lekérdezést ír a Neo4j-ben tárolt üzemi rétegre (`cypher_rag_lines`). A prompt Benedek sémaszövegét (a csúcscímkéket, a kapcsolatokat és a tulajdonságokat), egy rövid, általános P&ID-olvasási bevezetőt (primer) és lekérdezési formaszabályokat tartalmaz: korlátlan hosszúságú változó utat nem szabad kötni, halmazhoz különböző végpontokat kell kérni, útvonalhoz a legrövidebb utat, elérhetőséghez létezésvizsgálatot a `line_to` mentén. A lekérdezés csak olvashat, 30 s időkorláttal és legfeljebb 1000 sorral fut. Ha a lekérdezés hibát ad, egy javítókörben a modell megkapja az adatbázis hibaüzenetét, és újraírhatja. Egy programozott ellenőrzés emellett azt is megnézi, hogy a kérdésben szereplő tag-ek léteznek-e. A sorokból a közös válaszlépés fogalmaz.

Az elfogadási szabály. A Cypher-szint válaszát a kaszkád akkor fogadja el, ha az önjelzések mind rendben vannak: a lépés válaszolt, nem tartózkodott („nincs benne" válasz helyett érdemben felelt), a lekérdezés adott sort, és az eredmény nem csonka (nem érte el a sorkorlátot). Egyik jel sem igényli a helyes válasz ismeretét, ezért a szabály a teszten is alkalmazható, és nincs illesztett paramétere. Ha bármelyik jel hiányzik, a kérdés a második szintre kerül.

Második szint: gráfeszköz-ágens. Az ágens (`graph_agent_lines`) üzemi primitíveket hív lépésenként, legfeljebb 40-szer. Minden lépésben egy JSON-akciót ír (egy primitív hívását az argumentumaival, vagy a befejezést), a primitív eredményét szövegesen kapja vissza, és a végén az összegyűjtött megfigyelésekből ugyanaz a közös válaszlépés fogalmaz, amely a Cypher-szint válaszát is írja. A közös válaszlépés szabályai a hangoláskor szigorodtak: egy berendezés- és szeleplistában csak ezek szerepelhetnek, a kiinduló elem nem kerül a listába, és ha egy kérdezett tag nem létezik, a válasz „nincs benne".

Címke szerinti indítás. A használt kaszkád egyetlen kivételt tesz: az „első szelep" típusú kérdések (a felfelé vagy lefelé eső első szelep, `UPSTREAM_TO_FIRST_VALVE`, `DOWNSTREAM_TO_FIRST_VALVE`) rögtön az ágensnél indulnak. Ennek oka Benedek elemzése szerint az, hogy az ilyen kérdésekre írt Cypher-lekérdezések jellemzően az összes felfelé eső szelepet kérik az első helyett; amíg ezek lejárnak, a kaszkád az ágenshez fordul, ha viszont gyorsan lefutnának, rossz választ adnának, amelyet a kaszkád elfogadna. Benedek élő kaszkádja a címke szerinti útválasztást elutasítja, ezért két kaszkádot futtatunk (a teljeset és egy csak-ágenseset), és a kérdéscsalád címkéje alapján választunk köztük. A Cypher-szint és a válaszlépés alacsony, az ágens magas gondolkodási szinttel fut (a GLM-modellhez Benedek csak alacsony szintű beállítást rögzített, ezért ennek magas szintű másolatát használjuk).

A kaszkád elfogadási szabályait, paramétereit és promptjait változatlanul vettük át; kivétel a kérdésenkénti időkorlát, amelyet a hangoláskor növeltünk (lásd alább). A tiereket a saját mérőprogramunkkal raktuk össze Benedek stratégiaépítőjével, mert az ő élő kaszkádja a tokenhasználatot nem méri; a stratégiák és a promptok ugyanazok.

Az ágens primitívjei

Benedek ágense hat általános, egymással komponálható primitívet hív: a szűrőnek megfelelő elemek keresését (`find`), a szomszédokat (`neighbours`), a bejárást (`traverse`), az

utat két elem között (`path`), egy korábbi eredmény szűkítését (`filter`) és összesítését (`aggregate`). A primitívek a feloldó kimenetéből a memóriában épített, a betöltött üzemi réteggel egyező elemgráfon futnak, ezért az ágens nem látja a laphatárokat, csak az üzemet; az eredmények hivatkozást kapnak, így egy korábbi lépés eredménye a következő bemenete lehet. A tervezési döntés lényege, hogy az eszközök száma a gráfsémából következik, nem a kérdéstípusokból: egy új kérdéstípus egy új kompozíció, nem egy új eszköz (részletesen: Függelék E).

Az adatlapoldali A2

A plantdocs A2-módszere egy eszköz egy kérdésére a következőképpen válaszol. A rendszerüzenet a plantdocs gráfmodelljének leírását, az eszköz gráfjának leltárát és a forrásszakaszok katalógusát tartalmazza. Egy LLM egyetlen csak olvasó, széles keresésű Cypher-lekérdezést ír, és legfeljebb három forrásszakaszt választ, amelyek minden sora a lekérdezés sorai mellé kerül; hibás, üres vagy túl sok sort adó lekérdezésnél egy javítókör következik. Ezzel párhuzamosan a kérdés beágyazásvektorához legközelebbi 30 adatpont kerül elő a gráf vektorindexéből. A válaszíró csak a megtalált tényekből fogalmaz: a közvetlen válasz után az értékeket az egységükkel, a feltételeikkel és oldalhivatkozással adja meg, új tényt, értéket vagy eljárást nem tehet hozzá (részletesen: Függelék E).

Az összevont gráfban több eszköz gráfja van egyszerre, ezért néhány illesztésre volt szükség, a plantdocs kódjának módosítása nélkül. A leltár elé egy sor kerül, amely közli, hogy a kérdés melyik dokumentumról szól, és hogy minden csúcsot `document_id` szerint kell szűkíteni. A lekérdezés-futtató csak olyan lekérdezést enged, amely a dokumentum azonosítóját tartalmazza, és semmilyen írást nem enged. A vektoros keresés egy nagyobb jelöltkészletből csak a dokumentum csúcsait tartja meg. A másik lehetőség az lett volna, hogy kérdésenként újratöltjük az adatbázist a kérdezett eszköz gráfjával, ahogy a plantdocs alkalmazása teszi; ez az összevont gráfot tönkretenné, és az M1 számára értelmetlenné tenné az egész összevonást.

5.2 M1 — egy gráf, egy lekérdezés

Az M1 Benedek kaszkádja az összevont gráfon, két kiegészítéssel. A Cypher-szint sémaleírása az üzemi réteg mellett az adatlapréteget is tartalmazza: a `DESCRIBED_BY` kapcsolatot, a plantdocs A2-promptjának gráfmodelljét és lekérdezési szabályait, valamint a tag-ek dokumentumonkénti listáját. Így egyetlen lekérdezés követheti a rajzot és olvashatja az adatlapot, például:

```
MATCH (p:PlantItem {tag: 'P-1-3'})-[:DESCRIBED_BY]->(dev)
MATCH (d) WHERE d.document_id = dev.document_id AND d:plm__PCA_100029511
  AND toLower(d.plm__PCA_100029521) CONTAINS 'casing pressure'
RETURN d.plm__PCA_100029521 AS label, d.plm__PCA_100029522 AS value, d.schema__pageStart
AS page
```

A sémakiegészítés felépítését a Függelék E, elejét a Függelék C mutatja; a plantdocs lekérdezési szabályai a második fejlesztési iterációban kerültek be.

Az ágens egy új primitívet kap: a `datasheet_fact(tag, question)` a tag-ből feloldja az eszközt, és a plantdocs A2-logikájával kérdezi meg az adatlapját. A primitív Benedek

eszközregiszterébe kerül, ezért az ágens promptjában ugyanúgy jelenik meg, mint a többi primitív. A regisztráció csak az ágens felépítésének és a kérdés megválaszolásának idejére szól, így Benedek kódja és a többi futás eszközkészlete nem változik. Az eredmény Benedek eredményosztályainak felületét követi (szöveges megjelenítés hosszkorláttal), és tartalmazza a dokumentum azonosítóját, a tag-et, az első forrásoldalakat és az A2 válaszát. Ismeretlen vagy adatlap nélküli tag-re a primitív nem hibát dob, hanem szövegesen közli, hogy a tag-hez nem tartozik adatlap, így az ágens folytathatja a munkát. Az eszköz leírása arra kéri az ágenst, hogy a rajzon nem látható tulajdonságokhoz ezt használja, és előbb a rajzon keresse meg a tag-et.

Az M1 útvonala a lefutott szintekből és az adatlapi hívásokból áll (például csak a Cypher-szint, vagy a Cypher-szint, az ágens és egy `datasheet_fact:FT-2-2` hívás). A válaszlépés Benedek változatlan válaszpromptja; az M2-ben és az M3-ban bevezetett válaszszabályt (részválasz, a dokumentumon túli tanács tilalma) az M1 ezért nem kapta meg, mert az a kaszkád része, amelyet nem módosítunk.

5.3 M2 — router

Az M2 egy ágens két eszközzel: a `drawing(question)` Benedek kaszkádját hívja a rajzra (változtatás nélkül), a `datasheet(tag, question)` a tag-ből feloldott eszköz A2-jét. Az ágens minden lépésben egy JSON-objektumot ír (eszközhívást vagy végső választ), legfeljebb 16 lépésben; ha a keret elfogy, a router egy utolsó hívásban köteles végső választ adni a már összegyűjtött eredményekből (lásd alább, a hangolásnál). Ha a kérdés egy elemet a helyével vagy szerepével nevez meg, előbb a rajzeszközt kérdezi meg a tag-ről, utána az adatlapot.

A protokoll. A router minden lépésben pontosan egy JSON-objektumot ír (rajzeszköz-hívás, adatlap-hívás vagy végső válasz a „nincs benne” jelzővel); a program végrehajtja a hívást, és az eredményt egy új üzenetben adja vissza. A router alacsony gondolkodási szinttel fut; a rendszerüzenetet a Függelék B, a hibakezelést a Függelék E írja le.

A válaszszabály. A második fejlesztési iterációtól az M2 és az M3 egy közös válaszszabályt kap: ha a dokumentumok a kérdésnek csak egy részére adnak választ, a módszer ezt a részt adja meg, és megnevezi a hiányzó részt; a „nincs benne” jelzőt csak akkor állítja, ha semmi releváns nincs; tanácsot, eljárást, okot vagy értéket, amelyet a dokumentumok nem mondanak ki, nem tehet hozzá. A szabály után a dev halmazon az M2 és az M3 hallucinációi eltűntek (6. fejezet).

Tervezési döntések. Az M2-ben mindkét rendszer abban a beállításban fut, amelyre a szerzője hangolta, és a két rendszer egymásról nem tud; az összekötést a router végzi a kérdés szövegének értelmezésével. Ennek két következménye van. A rajzeszköz szabad szöveges kérdést kap (`FREE_TEXT`), a kérdéscsalád címkéjét nem, ezért a kaszkád az „első szelep” kérdéseket is a Cypher-szinten kezdi, és a rajzeszköz válaszának alakja eltérhet attól, amit a kérdés eredetileg kért (taghalmaz, út); ezt a hibát a 6. fejezetben egy szabályozókörös kérdésen látjuk is. A másik következmény, hogy az összetett kérdés legalább két egymás utáni eszközhívást igényel, amelyek mindegyike maga is több LLM-hívás, ezért a késleltetés nagyobb.

5.4 M3 — gráf nélküli baseline

Az M3-ban nincs gráf. A kontextus a teljes rajz Benedek ContextRAG-szerializálásával (ez a ChatP&ID-módszer: az egész rajz a promptban) és a hét adatlap PDF-szövegéből vektoros kereséssel kiválasztott 12 legközelebbi darab, ugyanazzal a beágyazómodellel, amelyet a plantdocs is használ (`qwen/qwen3-embedding-8b`, 512 dimenzió). A válasz egyetlen hívás magas gondolkodási szinttel, ahogy Benedek ContextRAG-karja is fut.

A szerializált rajz a rajzolt előfordulások gráfja: lapról lapra minden szimbólum egy csúcs a tag-jével és az osztályával, az élek a csövek és a jelek, a lapokat a lapközi csatlakozók kötik össze. Az M3 modellje tehát maga követi a csatlakozókat a laphatáron át; az összevont üzemi réteget nem látja. Az adatlap-darabok a PDF-ek szövegrétegéből készülnek oldalanként, bekezdéshatáron vágva, korlátozott hosszúságú darabokra; minden darab a dokumentum azonosítóját és az oldalszámot hordozza. A prompt tartalmazza a tag → adatlap táblát is, hogy a modell tudja, melyik darab melyik elemhez tartozik.

A baseline-t szándékosan erősnek választottuk: a négylapos prototípus-üzem rajza kicsi, így a teljes rajz elfér a promptban, és a ChatP&ID mérése szerint egyetlen lapon éppen ez a módszer nyert[4]. Az adatlapoldalon a szövegdarabok vektoros keresése a legegyszerűbb, gráf nélküli visszakeresés; ha az M3 felveszi a versenyt a két gráfos módszerrel, az arra utal, hogy ezen a méreten a gráf előnye nem a visszakeresésben, hanem legfeljebb a feltételek megőrzésében jelentkezhet.

5.5 Hangolás a fejlesztési üzemeken

Az első prototípust a négylapos üzemen fejlesztettük (Függelék G). A 30 és 50 lapos fejlesztési üzemen három változtatásra volt szükség; az M3-at nem hangoltuk.

- **M2: nagyobb lépéskeret és kötelező végső válasz.** A router eszközkerete 6-ról 16 lépésre nőtt, és ha a keret elfogy, a router egy utolsó hívásban köteles végső választ adni. A 30 lapos üzemen ugyanis a 24 több adatlapos válaszból 10 azért maradt üres, mert a router elérte a lépéskorlátot.
- **M1: egy új sémaszabály.** Sok M1-válasz csak részleges volt, ezért a Cypher-szint sémaleírása egy szabállyal bővült: az adatlapi tényeket tágan kell szűrni, és minden értéket a feltételével és az oldalszámával együtt kell visszaadni.
- **Kaszkád: hosszabb időkorlát.** A kaszkád kérdésenkénti időkorlátja 120 s-ről 240 s-re nőtt, mert egy kérdés túllépte a korábbit.

A hangolás után a módszereket befagyasztottuk, és a 100 és 1000 lapos tesztüzemen ezen a változaton nem változtattunk.

5.6 A módszerek várható erősségei és gyengeségei

Az alábbiakban a módszerek felépítéséből következő, minőségi várakozásokat foglaljuk össze; a 6. fejezet eredményeit ezek fényében érdemes olvasni.

M1

Az M1 erőssége, hogy egy összetett kérdés elvben egyetlen lekérdezéssel megválaszolható: a rajzi lépés és az adatlapi tény ugyanabban a mintában szerepel, így nincs szöveges átadás két rendszer között, és a válasz minden eleme visszakövethető egy csúcsig. Az ágensszint a

`datasheet_fact` primitívvel ugyanazt az A2-logikát éri el, mint az M2, tehát ahol a Cypher-szint elakad, ott az M1-nek az M2-höz hasonló képességei vannak.

A gyengeségei ugyanebből fakadnak. A Cypher-író modellnek egyszerre két szókincset kell kezelnie: Benedek olvasható osztálycímkéit és a plantdocs előtagos, számkódos szabványos neveit. A sémaszöveg ettől hosszú, ami minden kérdés költségét növeli, akkor is, ha a kérdés csak a rajzról szól. A legnagyobb kockázat az elfogadási szabály: a rajzi kérdéseken a nem üres, nem csonka eredmény jó jelzés, egy adatlapi kérdésnél viszont egy szűk lekérdezés is adhat nem üres, de hiányos sort (például egy értéket a feltétele nélkül), amelyet a kaszkád elfogad, és az ágens nem kap esélyt. Végül a válaszlépés a rajzi kérdésekre készült, és nem tartalmazza az adatlapi válaszíró szabályait (feltételek kiírása, oldalhivatkozás).

M2

Az M2 erőssége a modularitás: mindkét rendszer abban a beállításban fut, amelyben a szerzője mérte, az adatlapi válasz a plantdocs teljes A2-láncán megy át (szakaszválasztás, vektoros keresés, a feltételeket kiíró válaszíró), és egy új dokumentumtípus egy új eszközzel bővíthető. Az útvonal átlátható: a naplóból látszik, melyik eszközt milyen kérdéssel hívta a router.

A gyengesége az illesztésben van. A két rendszer szöveggel kommunikál, így a rajzeszköz válaszának alakja elcsúszhat, és a router félreértheti; az összetett kérdés helyes felbontása is a routeren múlik. Mivel az eszközhívások egymás után futnak, és mindegyik maga is több LLM-hívás, a késleltetés a három módszer közül várhatóan itt a legnagyobb.

M3

Az M3 erőssége az egyszerűség: egyetlen hívás, nincs lekérdezés, amely elromolhat, és nincs elfogadási szabály, amely rossz választ engedhet át. Egy kis rajzon a teljes gráf elfér a promptban, és a modell mindent lát; az adatlapi kérdéseknél a kérdés szavaihoz közeli szövegrészek gyakran éppen a keresett táblázatot tartalmazzák.

A gyengesége a méret és a szerkezet. Benedek mérései szerint a teljes rajz promptba tétele a lapszámmal romlik és drágul, egy határ fölött pedig nem is fér el (2. fejezet), ezért az M3 előnye nagyobb üzemen nem várható. Az adatlap-darabok a táblázatokat szövegként, a fejlécektől elszakítva tartalmazhatják, a feltételek és a lábjegyzetek nem explicitek, és a vektoros keresés egy több eszközt érintő kérdésnél elhanyagolhatja valamelyik eszköz adatlapját. Az M3 a forrásoldalakat csak darabszinten tudja megadni.

5.7 Költség és mérés

Minden LLM-hívás egy közös, cache-elt és naplózott rétegen megy át. A cache miatt az ismételt futás nem fizet újra; minden élő hívás költsége a közös naplóba kerül, és a teljes keret 20 USD (18 USD felett új mérés nem indul). Kérdésenként mérjük a tokeneket, a szolgáltató által jelentett költséget (cache-ből érkező válasznál az eredeti hívás árát), a késleltetést és az útvonalat (melyik szint vagy eszköz kellett a válaszhoz).

6. Kiértékelés

Ebben a fejezetben a három módszer mérését mutatjuk be. Az első prototípust a négylapos kis üzemen építettük fel; a közös rendszert ezután két fejlesztési üzemen, a pontosan 30 és 50 lapos üzemen hangoltuk, majd befagyasztottuk, és két olyan üzemen mértük meg, amelyet a hangolás során nem használtunk: a pontosan 100 és 1000 lapos tesztüzemen. A tesztüzemeken minden kérdés tesztkérdés.

6.1 Beállítás

Mindhárom módszert mindkét kérdés-válaszoló modellel futtattuk: `z-ai/glm-5.3-flash` (a továbbiakban GLM) és `openai/gpt-6-luna` (Luna), OpenRouteren át, a rajzoldali rendszer rögzített beállításaival (a GLM-et alacsony és magas gondolkodású változatban). A pontosság a helyes válaszok aránya; a konfidenciaintervallum 10 000 ismétléses, seedelt percentilis-bootstrap, a módszerpárok különbségét ugyanazokon a kérdéseken, párosítva számoltuk. A teljes projekt fizetős költése 18,8923 USD volt; ebből a fejlesztési üzemek mérése és a hangolás az 50 lapos üzemen 1,2599 USD, a két tesztüzem mérése 2,8544 USD.

Az összevont gráf mérete az üzemmel nő, az adatlapréteg változatlan (27 615 csúcs, 103 366 él): a 30 lapos üzemen 274 üzemi csúcs, 931 üzemi él és 174 `DESCRIBED_BY` kapcsolat, az 50 lapos üzemen 498, 1713 és 324, a 100 lapos üzemen 1018, 3515 és 644, az 1000 lapos üzemen 10 277, 35 603 és 6641; az 1000 lapos üzem betöltése 80,7 s-ig tartott.

6.2 Az első prototípus a kis üzemen

A kis üzem 20 kérdéses fejlesztési halmazán négy iterációban alakítottuk ki a módszerek első változatát; a 40 kérdéses teszthalmazon a legjobb az M2 volt GLM-mel (31 helyes), a gráf nélküli M3 30, az M1 25 helyes választ adott. A részleteket a Függelék G írja le.

6.3 Hangolás a fejlesztési üzemeken

A prototípus módszereit a 30 és az 50 lapos fejlesztési üzem mind a 60 kérdésén és 12 több adatlapos kérdésén futtattuk. Az alábbi táblázat a hangolás előtti eredményt mutatja; a gráf nélküli M3-at nem hangoltuk, ezért csak a 30 lapos üzemen futott.

5. táblázat: A fejlesztési üzemek eredménye a hangolás előtt (helyes válaszok, 60 kérdés; (e): 12 kérdés)

Módszer	Modell	30 lap	50 lap	(e), 30 lap	(e), 50 lap
M1	GLM-5.3 Flash	45	47	2	3
M2	GLM-5.3 Flash	50	52	3	5
M3	GLM-5.3 Flash	41	–	0	–
M1	GPT-6 Luna	49	44	2	0
M2	GPT-6 Luna	46	43	2	4
M3	GPT-6 Luna	38	–	0	–

A hibák elemzése három javítást hozott (az 5. fejezet írja le őket). Az M2 a több adatlapos kérdéseknél minden elem adatlapját külön lekérdezi, és a hatlépéses eszközkeret a 30 lapos üzemen a 24 ilyen válaszból 10-et, az 50 lapos üzemen 8-at üresen hagyott; a keret ezért 16

lépés lett, és ha mégis elfogy, a módszer egy utolsó hívásban köteles a már összegyűjtött eredményekből válaszolni. Az M1 hibáinak nagy része hiányos feltételű részválasz volt, ezért a Cypher-szint sémaleírása egy szabállyal bővült (adatlapi tényeknél tág szűrés, minden érték a feltételével és az oldalával). Végül egy sok adatlap-hívást igénylő kérdés a 120 s-os időkeretben válasz nélkül maradt, ezért a keret 240 s lett. A javítást az 50 lapos üzem több adatlapos kérdésein ellenőriztük: az M2 üres válaszai modellenként 4-ről 2-re csökkentek, a teljesen helyes válaszok száma az M2-nél GLM-mel 5-ről 6-ra nőtt, Lunával 4 maradt, az M1-nél GLM-mel 3 maradt, Lunával 0-ról 2-re nőtt. Ezután a módszereket befagyasztottuk.

6.4 A gráf nélküli módszer az 1000 lapos üzemen

Az M3 a teljes rajzot szövegként, Benedek szerializálásával kapja meg a promptban. Ez a szöveg a 100 lapos üzemen 474 716, az 1000 lapos üzemen 4 793 987 karakter. Egy korábbi, 98 lapos próbaüzemen végzett mérésben egy ekkora rajz kérdésenként átlagosan kb. 194 ezer prompttokent jelentett, ezzel az aránnyal az 1000 lapos rajz kérdésenként kb. 1,96 millió token lenne. Ez meghaladja mindkét modell kontextusát (GLM: 1 048 576, Luna: 1 050 000 token az OpenRouter modellistája szerint), ezért az M3-at az 1000 lapos üzemen nem lehetett lefuttatni; ott csak a két gráfos módszert mértük. Ez maga is eredmény: a teljes rajz promptba tétele egy valódi méretű üzemnél nem csak drága, hanem kivitelezhetetlen, a gráfos módszerek viszont ugyanúgy működnek.

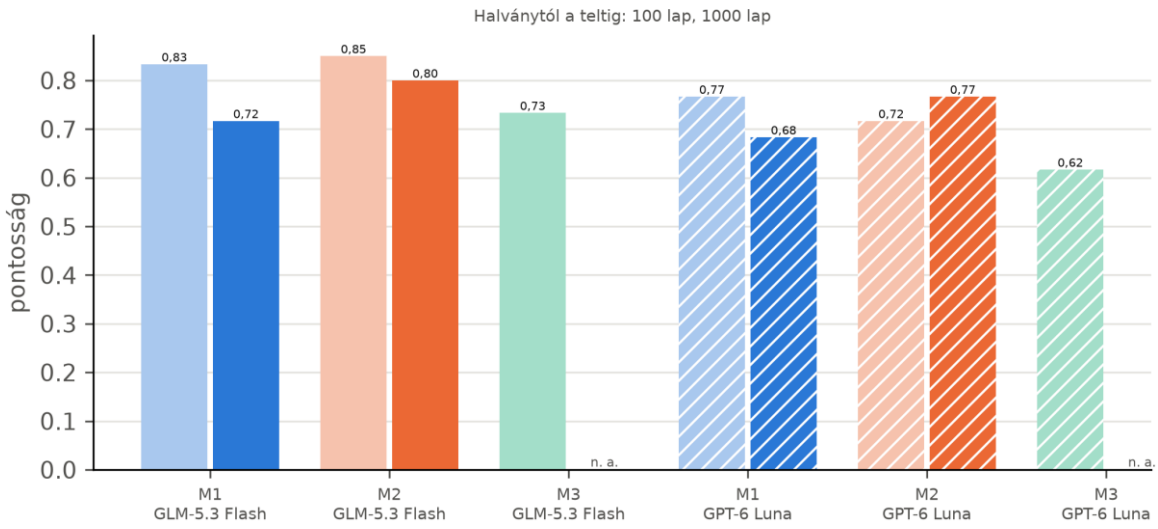
6.5 Eredmény a két tesztüzemen

A hangolt, befagyasztott módszereket mindkét tesztüzem 60 kérdésén (családonként 15) mindegyik párosításon egyszer futtattuk. Az eredményt az alábbi táblázat és ábra mutatja.

6. táblázat: Eredmény a két tesztüzemen párosításonként és családonként (60-60 kérdés; az 1000 lapos üzemen az M3 nem futtatható)

Módszer	Modell	Lap	Helyes	Részben	Rossz	Halluc.	Pontosság [95% CI]	(a)	(b)	(c)	(d)
M1	GLM-5.3 Flash	100	50	4	6	0	0,833 [0,733–0,917]	13	13	9	15
M1	GLM-5.3 Flash	1000	43	6	10	1	0,717 [0,600–0,833]	12	12	5	14
M2	GLM-5.3 Flash	100	51	5	4	0	0,850 [0,750–0,933]	13	12	11	15
M2	GLM-5.3 Flash	1000	48	4	8	0	0,800 [0,700–0,900]	12	10	11	15
M3	GLM-5.3 Flash	100	44	2	14	0	0,733 [0,617–0,833]	11	8	10	15
M1	GPT-6 Luna	100	46	10	4	0	0,767 [0,650–0,867]	14	9	8	15

Módszer	Modell	Lap	Helyes	Részben	Rossz	Halluc.	Pontosság [95% CI]	(a)	(b)	(c)	(d)
M1	GPT-6 Luna	1000	41	14	5	0	0,683 [0,567–0,800]	14	6	6	15
M2	GPT-6 Luna	100	43	10	7	0	0,717 [0,600–0,833]	10	8	10	15
M2	GPT-6 Luna	1000	46	9	5	0	0,767 [0,650–0,867]	12	10	9	15
M3	GPT-6 Luna	100	37	9	14	0	0,617 [0,483–0,733]	13	3	9	12



4. ábra: A pontosság a 100 lapos (halvány oszlop) és az 1000 lapos tesztüzemen (telt oszlop), párosításonként; az 1000 lapos üzemen az M3 nem futtatható (n. a.).

A 100 lapos tesztüzemen GLM-mel az M2 a legjobb párosítás (51 helyes a 60-ból), szorosan mögötte az M1 (50; a különbség −0,017, 95% CI −0,083 és +0,050 között, nem szignifikáns). GLM-mel az M2 szignifikánsan jobb a gráf nélküli M3-nál (+0,117, +0,033 és +0,217 között), az M1 a határon (+0,100, 0,000 és +0,217 között); Lunával az M1 szignifikánsan jobb az M3-nál (+0,150, +0,033 és +0,267 között), az M2 nem (+0,100, −0,033 és +0,233 között).

Az 1000 lapos tesztüzemen is az M2 GLM-mel a legjobb párosítás (48 helyes), utána az M2 Lunával (46), az M1 GLM-mel 43, Lunával 41 helyes választ adott; az M1 és az M2 különbsége egyik modellel sem szignifikáns (GLM-mel −0,083, 95% CI −0,217 és +0,033 között, Lunával −0,083, −0,183 és +0,017 között). A tízszer nagyobb rajzon az M2 GLM-mel 3 kérdéssel, az M1 GLM-mel 7, Lunával 5 kérdéssel kevesebbet oldott meg, mint a 100 lapos üzemen, az M2 Lunával 3-mal többet; a visszaesés főleg az összetett (c) családból jön (az M1 GLM-mel 9 helyett 5 helyes).

A hangolás hatása a tesztüzemen kicsi: a 100 lapos üzemen a hangolás előtti módszerekkel is lefutott ugyanez a mérés, és a helyes válaszok száma párosításonként legfeljebb hárommal tért el (az M2 GLM-mel 54 és 51, Lunával 42 és 43; az M1 GLM-mel 50 és 50,

Lunával 45 és 46). A hangolás tehát a fejlesztési üzemeken látott üres válaszokat csökkentette, a pontosságot a tesztüzemen mérhetően nem változtatta; ez a kis mintán belüli ingadozás nagyságrendje.

6.6 Költség

7. táblázat: Kérdésenkénti költség a két tesztüzemen (USD)

Módszer	Modell	100 lap	1000 lap
M1	GLM-5.3 Flash	0,00416	0,00592
M2	GLM-5.3 Flash	0,00451	0,00866
M3	GLM-5.3 Flash	0,00757	–
M1	GPT-6 Luna	0,01089	0,01272
M2	GPT-6 Luna	0,00248	0,00385
M3	GPT-6 Luna	0,02501	–

A gráfos módszereknél a modellhez csak a lekérdezés eredménye jut el, ezért a kérdésenkénti költségük sokkal lassabban nő, mint a rajz: a 100 és az 1000 lapos üzem között az M2 GLM-mel 0,00451-ről 0,00866 USD-re, Lunával 0,00248-ról 0,00385 USD-re, az M1 GLM-mel 0,00416-ról 0,00592 USD-re, Lunával 0,01089-ről 0,01272 USD-re nőtt, vagyis a tízszer nagyobb rajzon legfeljebb a kétszeresére; ennek okát részletesen nem vizsgáltuk. Az M3 a 100 lapos üzemen GLM-mel 0,00757, Lunával 0,02501 USD-be került kérdésenként, Lunával tízszer annyiba, mint az M2 (0,00248 USD), kevesebb helyes válasszal; a fejlesztési üzemek közül a 30 lapos üzemen még 0,00206, illetve 0,00731 USD volt, az 1000 lapos rajz pedig már nem fér a modellek kontextusába. A késleltetést a tesztüzemeken nem hasonlítjuk össze, mert a válaszok egy része (az M3 válaszai és az M2 rajzi hívásainak egy része) a korábbi futások eltárolt válaszaiból jött.

6.7 Több adatlapos kérdések

Mindkét tesztüzemen lefuttattuk a 12 kérdéses (e) családot is, amelyben a rajzon egy utat, egy elemhalmazt vagy két topológiai lépést kell bejárni, és kettő–hat különböző adatlapot kell összevetni (például: „Along the process flow path from P-2-1 to E-38-1, which item has the lowest maximum allowable pressure according to its datasheet?”, a 100 lapos üzemen). A referenciát program rakta össze a generátor hibátlan gráfjából és a PDF-ekből gyűjtött tényekből.

8. táblázat: A több adatlapos kérdések eredménye a két tesztüzemen (12-12 kérdés)

Módszer	Modell	Helyes, 100 lap	Részben, 100 lap	Helyes, 1000 lap	Részben, 1000 lap
M1	GLM-5.3 Flash	5	2	1	3
M2	GLM-5.3 Flash	4	4	3	5
M3	GLM-5.3 Flash	0	3	–	–
M1	GPT-6 Luna	0	4	0	5
M2	GPT-6 Luna	0	7	1	8
M3	GPT-6 Luna	0	3	–	–

A gráf nélküli M3 a 100 lapos üzemen egyik kérdésre sem adott teljesen helyes választ. GLM-mel a gráfos módszerek 4–5 kérdést oldottak meg teljesen (az M1 és az M3 különbsége +0,417, 95% CI +0,167 és +0,667 között, az M2-é +0,333, +0,083 és +0,583 között), Lunával egyet sem, csak részben (4, illetve 7 kérdés). Az 1000 lapos üzemen ezek a kérdések még nehezebbek lettek: teljesen helyes választ az M2 GLM-mel 3, Lunával 1, az M1 GLM-mel 1, Lunával egy kérdésre sem adott. Az M2 üres válaszai a hangolás után is megmaradtak, de ritkábbak: a 100 lapos üzemen modellenként 2 (a hangolás előtt 3), az 1000 lapos üzemen GLM-mel egy sem, Lunával 1.

6.8 Hibaelemzés

Téves „nincs benne" válaszok. A gráf nélküli módszer jellemző hibája a hamis lemondás, főleg a több adatlapos kérdéseken: az M3 GLM-mel a 30 lapos fejlesztési üzemen 7, a 100 lapos tesztüzemen 5 esetben állította, hogy a dokumentum nem tartalmazza a választ. Ez veszélyesebb, mint amilyennek látszik, mert a „nincs benne" a megválaszolhatatlan kérdéseken helyes, így a (d) családon az M3 jól szerepel, miközben a megválaszolható kérdéseken elbukik.

Az M1 egyetlen lekérdezése. Az M1 hibáinak nagy része „részben helyes" válasz: a Cypher-szint egy szűk lekérdezéssel talál egy releváns adatpontot, a kaszkád elfogadja a nem üres eredményt, és a válaszból kimarad egy feltétel vagy egy második érték (példa a Függelék G-ben). Az elfogadási szabály a rajzi kérdésekre készült, ahol a nem üres sorhalmaz jó jelzés; adatlapi kérdésnél a hiányos sor is nem üres.

Az M2 eszközhívásai. Az M2 a rajzeszköznek szabad szöveges kérdést ad, ezért a válasz alakja elcsúszhat (a kis üzemen például szabályozókör helyett a szelepét adta meg, Függelék G), és a több adatlapos kérdéseknél többször üres választ adott, mielőtt minden elem adatlapját lekérdezte volna (lépéskorlát vagy feldolgozási hiba). A hangolás után ezek ritkábbak lettek (a 100 lapos tesztüzemen modellenként 2).

A referencia korlátja. Az (e) család referenciája óvatos: ha valamelyik elemnek van olyan változata, amelyhez az adatlap nem ad számot, a „legkisebb" elem nem dönthető el, és a bíráló ilyenkor a konkrét elemet megnevező választ is rossznak ítéli. A referenciák általános korlátait a

1. fejezet foglalja össze.

7. Demó és esettanulmány

A mért számok azt mutatják meg, milyen gyakran helyes egy módszer, azt nem, hogyan jut el a válaszig. Ebben a fejezetben bemutatjuk a webes demót (a dolgozat ötödik hozzájárulását), majd két képernyőképen végigkövetünk két kérdést; a próba további négy kérdését a Függelék F mutatja be. A képek a demó próbáján készültek: hat kérdést tettünk fel a futó demón, az (a), (a), (b), (c), (d) és (e) családból, az M1, M2 és M3 módszerrel, GLM-mel; mind a hat kérdésre érdemi válasz jött. A próba nem pontossági mérés, a válaszokat nem pontoztuk.

7.1 A demó felépítése

A demó egy helyi webalkalmazás: egy webszerver és egyetlen weboldal. A szerver ugyanazokat a módszerobjektumokat építi fel, amelyeket a mérés használ (az első kérdésnél, utána újrahasznosítja őket), így a demó válaszai pontosan azon az úton születnek, amelyet az 5. fejezetben leírtunk. A korábban feltett kérdésekre a demó a már eltárolt válaszokkal is tud felelni, új, fizetős modellhívás nélkül.

Az oldal két részre oszlik.

- **Bal oldal: a dokumentum.** Alapállapotban a P&ID látszik, a generátor rajzexportjából: a fejléc megnevezi a korpuszt (a plantgraph 6-os seedű üzeme), és lapváltó gombokkal a négy lap között lehet lépkedni; a teljes rajz PDF-ben is megnyitható. Ha a felhasználó egy válasz adatlapi forrásoldalára kattint, a bal oldalon az adatlap adott oldala jelenik meg képként, a fejlécben a dokumentum azonosítójával és oldalszámával, valamint egy hivatkozással, amely a PDF-et ennél az oldalnál nyitja meg.
- **Jobb oldal: a kérdés és a válasz.** Egy szövegmezőbe kerül a kérdés; alatta választható a módszer (M1, M2 vagy M3, egy rövid leírással), a modell (`z-ai/glm-5.3-flash` vagy `openai/gpt-6-luna`) és a válaszalak (az 5. fejezetben felsorolt alakok közül). Az alapértelmezés az M2 GLM-mel, mert ez a párosítás volt a legjobb a kis üzem tesztjén (31 helyes a 40-ből; ezt az oldal ki is írja) és a 100 lapos tesztüzemen is (51 helyes a 60-ból). A demó a kis üzemen fut, mert annak négy lapja a képernyőn jól áttekinthető.

A válasz alatt a demó megmutatja a válasz nyomait: a módszert, a modellt, a kimenetelt, a késleltetést, a tokenszámot, a költséget és a gyorsítótárból jött hívások számát. Ezután következik az útvonal címkék sorozataként (az M1-nél a tierek és az adatlapi primitívhívások, az M2-nél a router eszközhívásai, az M3-nál a kontextus fajtája), az adatlapi forrásoldalak kattintható oldalszámként, és egy lenyitható blokkban a lefutott lekérdezések. Végül a dev halmaz kérdései szerepelnek példaként, családonként csoportosítva. A teszthalmaz kérdései nincsenek a példák között.

A képernyőképeket egy program készítette: minden kérdést előbb az API-n tett fel, majd fej nélküli böngészővel lefényképezte az oldalt. Ezért a képeken látható késleltetés a gyorsítótárból kiszolgált válaszé; a valódi késleltetést a 6. fejezet adja.

7.2 Két kérdés végigkövetve

A Függelék F két kérdést lépésenként végigkövet a demóban: egy összetett kérdést, amelynél a router előbb a rajzeszközt, majd a távadó adatlapját kérdezi, és egy megválaszolhatatlan kérdést, amelynél a „nincs benne” válasz mellett a demó azt is megmutatja, mit tartalmaz az adatlap.

7.3 Tanulságok

A demó azt mutatja meg, amit a pontossági számok nem: az útvonal és a lekérdezések láthatók, így egy rossz válasz gyorsan felismerhető, és a forrásoldal egy kattintással a rajz helyén jelenik meg.

A demó korlátai: egy új kérdés élő LLM-hívásokba kerül, és másodpercekig tart (a gyorsítótár csak az ismétlést teszi ingyenessé); a demó egyetlen üzemet ismer, mert egy másik üzemhez a teljes összevont betöltést újra kellene futtatni; és angol nyelvű, mint a kérdéssor és a két forrásrendszer promptjai.

8. Összefoglalás

Két egyéni munkát kötöttünk össze: a P&ID-rajzokból épített üzemi gráfot és a gyártói adatlapokból épített, eszközönkénti ISO 15926 gráfokat. Egyetlen Neo4j-adatbázisba töltöttük a két réteget, és egy tagalapú kapcsolattal kötöttük össze őket. Három kérdés-válaszoló módszert építettünk fel egy négylapos prototípus-üzemen, a közös rendszert egy 30 és egy 50 lapos fejlesztési üzemen hangoltuk, majd befagyasztva két modellel megmértük egy 100 és egy 1000 lapos, a hangolás során nem használt tesztüzemen, üzemenként 60 kérdésen és 12, több adatlapot összevető kérdésen.

8.1 Válaszok a kutatási kérdésekre

RQ1. A rajzot és az adatlapot összekötő kérdések megválaszolhatók az összevont gráfon: az M1 GLM-mel 50 és 43, Lunával 46 és 41 helyes választ adott a 60-ból a 100 és az 1000 lapos tesztüzemen. A két rész meglévő rendszerét külön hívó irányító (M2) GLM-mel mindkét tesztüzemen a legjobb párosítás volt (51 és 48 helyes), de az M1-től való különbsége nem szignifikáns. A gráf nélküli M3-mal szemben a 100 lapos üzemen GLM-mel az M2, Lunával az M1 előnye szignifikáns.

RQ2. A gráfos módszerek kérdésenkénti költsége a tízszer nagyobb rajzon legfeljebb a kétszeresére nőtt (az M2 GLM-mel 0,00451 és 0,00866 USD a 100 és az 1000 lapos üzemen), a gráf nélküli M3 költsége viszont a 30 lapos fejlesztési üzem és a 100 lapos tesztüzem között a GLM modellel 0,00206-ról 0,00757 USD-re, a Luna modellel 0,00731-ről 0,02501 USD-re nőtt, és az 1000 lapos rajz már nem fér a modellek kontextusába, így ott az M3 nem futtatható. A gráfos módszerek pontossága a 100 és az 1000 lap között legfeljebb 7 kérdéssel csökkent (az M2 GLM-mel 51-ről 48-ra, az M1 GLM-mel 50-ről 43-ra, az M2 Lunával viszont 43-ról 46-ra nőtt). A gráf tehát nemcsak olcsóbbá, hanem nagy üzemen egyáltalán lehetővé teszi a kérdés-válaszolást; ez egybevág Benedek mérésével, amely szerint a teljes rajz promptba tétele a lapszámmal romlik és drágul. A hangolás a fejlesztési üzemeken látott üres válaszokat csökkentette, a tesztüzem pontosságát mérhetően nem változtatta.

RQ3. A rajzi (a) és a megválaszolhatatlan (d) kérdéseken minden módszer erős, a különbség az adatlapi (b), az összetett (c) és a több adatlapos (e) kérdéseken jön ki. A több adatlapos kérdések közül a gráf nélküli módszer a 100 lapos üzemen egyiket sem oldotta meg teljesen, a gráfos módszerek GLM-mel 4–5-öt, az 1000 lapos üzemen 1–3-at. Az M1 jellemző hibája a hiányos egylekérdezéses válasz, az M2-é a több adatlapos kérdéseken a lépéskorlát miatti üres válasz, a gráf nélküli módszeré a téves „nincs benne” jelzés.

8.2 Korlátok

- **Szintetikus üzemek.** Az eredmény öt szintetikus üzemre érvényes (4, 30, 50, 100 és 1000 lap), amelyeket Benedek generátora állított elő szabályok szerint (egységek, berendezések, csővezetékek, szabályozókörök, lapközi csatlakozók); a rajz nem valódi üzem rajza, és a hét adatlap sok tag-et ír le.

- **Modellkészítésű referenciák.** A (b)–(d) család referenciáit egy modell írta a PDF-ből, ember nem ellenőrizte őket; a bíráló is modell (vak, de nem ember), és ugyanazt a választ két bírálat egyszer eltérően ítélte meg.
- **Kis minta.** Üzemenként és családonként 15 kérdés, egyetlen futás; a konfidenciaintervallumok szélesek, és a pontosság üzemenként néhány kérdéssel ingadozik.
- **A rajzgenerátor egyszerűbb változata.** A generátor későbbi változata valósághűbb folyamatmintákat épít (például desztillációs oszlopot, reaktort, tolózárat, biztonsági szelepet, mérőórákat és leürítőket), és a rajzoldali rendszert erre hangolták. Mi az első, egyszerűbb változatot használtuk (szivattyúk, hőcserélők, néhány szeleptípus és szabályozókörök), mert csak abban lehet a generált berendezésfajtákat azokra korlátozni, amelyekhez van adatlapunk.

8.3 Továbblépés

A legközvetlenebb javítás az M1 elfogadási szabályának adatlapi kérdésekre szabott változata (például a feltétel és az oldalszám meglétének ellenőrzése), és egy olyan Cypher-szint, amely a rajzi lépés után az adatlapi részt az adatlapoldali rendszerre bízza. Hosszabb távon a mérést valódi rajzokon, szélesebb adatlapkészlettel és emberileg ellenőrzött referenciákkal kell megismételni.

Irodalomjegyzék

[1] AIChE ChEnected (2010): „P&ID Part 3 – Symbology Primer”.

[2] AIChE ChEnected (2010): „P&ID Part 4: Codes, Tags and Labels”.

[3] Alimin, Goldstein, Schulze Balhorn, Schweidtmann (2025): *Talking like Piping and Instrumentation Diagrams (P&IDs)*. arXiv 2502.18928.

[4] Alimin & Schweidtmann (2026): *GraphRAG for Engineering Diagrams: ChatP&ID Enables LLM Interaction with P&IDs*. arXiv 2603.22528; a folyóiratváltozat: AIChE Journal, doi 10.1002/aic.70540

[5] POSC Caesar Association: Arrowhead fPVN Handbook és Use Cases (`docs.posccaesar.org/libraries-and-standards/arrowhead-handbook/`).

[6] IOGP JIP36: CFIHOS (Capital Facilities Information Handover Specification), „Version 2.0”.

[7] B. A. Christensen (2020): *Preparation of Process Flow Diagrams and Piping and Instrumentation Diagrams*, HNF-64103 Rev. 0. Mission Support Alliance, US DOE (Hanford).

[8] DEXPI e.V.: DEXPI Specification 2.0 (benne a DEXPI P&ID Specification 1.4), 2025. október.

[9] DEXPI e.V. hír, 2026-03-23: a TotalEnergies a DEXPI szemantikus (BFO- és IDO-igazított) változatán dolgozik.

[10] Fan et al. (2026): GraphRAG-Router. arXiv 2604.16401.

[11] Gadekar, Sakhinana, Runkana (2026): *Grounded and Faithful P&ID Reasoning: Constraining Vision-Language Models with Recovered Evidence Graphs*. arXiv 2609.05880v1.

[12] A. Gjerver, R. Frostad, V. Barisic, M. Hodkiewicz, C. Woods, M. Fekete, A. B. Torjusen, J. W. Kluwer (2025): *Machine-interpretable Engineering Design Standards for Valve Specification*. arXiv:2510.01736, 2025-10-02. Kód és ontológia: `github.com/PCA-POSC-Caesar-Association/IDO_Use_Case_Valve_Ontology_Public` (CC BY-SA 4.0).

[13] Golden-Retriever (2024). arXiv 2408.00798.

[14] Graph-CoT. arXiv 2404.07103.

[15] Mohit Gupta, Chialing Wei, Thomas Czerniawski, Ricardo Eiris (2025): *PIDQA - Question Answering on Piping and Instrumentation Diagrams*. Machine Learning and Knowledge Extraction 7(2), article 39, doi 10.3390/make7020039.

[16] Hirtreiter, Schulze Balhorn, Schweidtmann (2022): *Towards automatic generation of P&IDs with AI*. arXiv 2211.05583; AIChE J. 70(1) e18259.

[17] HyperGraphRAG (2025). arXiv 2503.21322, NeurIPS 2025.

[18] HyperRAG (2026). arXiv 2602.14470, WWW 2026.

[19] IDO (Industrial Data Ontology), korábban ISO/TR 15926-14, most ISO 23726-3 (ISO DIS / FDIS 23726-3; „IDO v4.2 (FDIS, April 2026)”).

[20] Industrial Ontologies Foundry: IOF, release 202603 (BFO-alapú, MIT).

[21] ISO 10628-1:2014 (folyamatábrák és P&ID-k), hivatalos előnézet, 1–5.4.3. fejezet.

[22] Middleware / KBQA-Agent. arXiv 2402.14672.

[23] KG-Agent. arXiv 2402.11163.

[24] Knollmeyer, Caymazer, Grossmann (2025): Document GraphRAG. Electronics 2025, 14(11):2102.

[25] Kochański et al. (2026): „How Everllence Scaled P&ID Intelligence…”. AWS Industries blog, 2026-05-27.

[26] Kumar, Motiyani, Kasturi, Seefried, Movva, Ghosal (2026): *Enginuity: A Dataset and Benchmark for Vision-Language Understanding of Engineering Diagrams*. arXiv 2606.03410.

[27] Lopomo et al. (2026): *Leveraging LLMs and Object Detection for Automated KG Generation from Industrial Schematics*. Big Data Cogn. Comput. 10(7):210, doi 10.3390/bdcc10070210.

[28] Mani, Haddad, Constantini, Douhard, Li, Poirier (C3.ai) (2020): *Automatic Digitization of Engineering Diagrams Using Deep Learning and Graph Search*. CVPRW 2020.

[29] Neo4j `mcp-neo4j-cypher` (neo4j-contrib/mcp-neo4j), szoftver.

[30] RWTH Aachen AVT: OntoCAPE 2.0 (62 OWL-fájl, GPL).

[31] OPC Foundation: OPC UA for DEXPI (OPC 30250 v1.0).

[32] Energy Impact Center: OPEN 100 atomerőmű-terv P&ID-jei (www.open-100.com).

[33] POSC Caesar Association Reference Data Library: a régi `data.posccaesar.org/rdl/RDS…` és az új `rds.posccaesar.org/ontology/plm/rdl/` (equipment modul 0.9.1, CC BY-SA 4.0).

[34] Stürmer, Graumann, Koch (DLR) (2025): PID2Graph adatkészlet. Zenodo, doi 10.5281/zenodo.14803338, CC BY-SA 4.0.

[35] GitHub `shaaz1000/pid-graph-agent` (2026), szoftver, nem lektorált.

[36] Prusty, Agarwal, Iyenger (2025): *Advanced Integration of Discrete Line Segments in Digitized P&ID for Continuous Instrument Connectivity*. arXiv 2505.11976.

[37] process-intelligence-research/pyDEXPI, szoftver (AGPL-3.0).

[38] QUDT (qudt.org), Release 2.1, CC BY 4.0.

[39] schema.org, V30.0 (2026-03-19).

[40] Schulze Balhorn, Seijsener, Dao, Kim, Goldstein, Driessen, Schweidtmann (2025): *Rule-based autocorrection of Piping and Instrumentation Diagrams (P&IDs) on graphs*. arXiv 2502.18493.

[41] W3C: SOSA/SSN, W3C Recommendation, 2017. október 19.

[42] StarE (2020). arXiv 2009.10847.

[43] Structured-data-aware RAG for technical documents (2025). arXiv 2506.23136.

[44] Stürmer, Graumann, Koch (DLR) (2024): *From Engineering Diagrams to Graphs: Digitizing P&IDs with Transformers*. arXiv 2411.13929, IEEE DSAA 2025.

[45] M. Toghraei (2019): *Piping and Instrumentation Diagram Development*. Wiley/AIChE (2. kiadás: ISBN 9781394368587).

[46] Xiang et al. (2025): GraphRAG-Bench, „When to use Graphs in RAG". arXiv 2506.05690v3.

[47] Baikai Zhu, Samuel Duong, Javal Vyas, Mehmet Mercangoez (2026): *From P&ID Drawings to Process Graphs: A Multimodal Language Model Approach*. arXiv 2607.19568.

Függelék

A. Példakérdések családonként (dev halmaz)

9. táblázat: Két példakérdés családonként a dev halmazból, a referenciaválasz rövidítve

Család	Kérdés	Referenciaválasz (rövidítve)
(a)	Can process flow reach P-2-2 from P-1-1?	no
(a)	Which valve is actuated by control loop PIC-2-1?	PV-2-1
(b)	Regarding P-1-2 on the P&ID: What's the hottest liquid I can put through it?	160 °F (71 °C) maximum liquid temperature (seal limit); the 250 psi max operating pressure applies only up to 100 °F (37 °C); consult the factory beyond.
(b)	Regarding FV-2-2 on the P&ID: What flow characteristics can I get: equal percentage, linear, quick opening?	Standard cages: quick-opening, linear or equal percentage. Whisper Trim, WhisperFlo and Cavitrol cages: linear only.
(c)	Consider the item directly upstream of CHV-2-1 in the process flow. What kind of shaft seal does it have, and what's the part number for a replacement seal?	Mechanical, self-lubricated seal: silicon carbide rotating and stationary faces, fluorocarbon (Viton or equivalent) elastomers, 316 stainless shaft sleeve and s
(c)	Consider the item directly upstream of CHV-2-2 in the process flow. What is a Fisher ET? Is it the valve body, and what kind of valve is it?	A sliding-stem, single-port globe-style control valve (body) with cage guiding, balanced valve plug and push-down-to-close plug action, for throttling or on-off
(d)	Regarding GV-1-2 on the P&ID: It hums but won't open. What could be wrong?	No troubleshooting in the datasheet. Related facts only: it needs at least 0.5 bar differential pressure to open; voltage must match the type plate (±10 %). The
(d)	Regarding PV-2-1 on the P&ID: The valve leaks through when shut. Is it the seat ring or the plug seal, and how do I check?	Not in the document (no troubleshooting).

B. Az M2 router rendszerüzenete

```
You answer questions about one industrial plant. You cannot see the documents
yourself; you have two tools:
- drawing(question): answers a question about the plant's P&ID drawing (which items exist
and
  their tags and types, what is connected to what, flow paths, upstream/downstream items,
  which instrument measures what, control loops, units, sheets).
- datasheet(tag, question): answers a question about the vendor datasheet of the item with
  this tag (its specification: pressures, temperatures, materials, sizes, options, ...).
  Items with a datasheet: <24 tags>.
If a question names an item by its position or role (e.g. "the valve downstream of P-1-
1"),
first ask the drawing tool for its tag, then ask the datasheet tool about that tag.
Each turn, reply with exactly one JSON object and nothing else:
{"tool": "drawing", "question": "..."}
{"tool": "datasheet", "tag": "...", "question": "..."}
{"final": {"answer": <answer or null>, "not_present": <true if the documents do not
contain
the answer, else false>}}
```

```
Answer only from the tool results. If the documents give only part of what is asked,
answer with that part and say which part is not stated; set not_present to true only when
they contain nothing relevant. Never add advice, procedures, causes or values that the
documents do not state.
Answer format: <Benedek answer-format sentence>
```

C. Az M1 Cypher-szintjének sémakiegészítése (eleje) és az új primitív leírása

```
DATASHEET LAYER (the same database also holds the datasheet of each tagged item)
- (:PlantItem {tag})-[:DESCRIBED_BY]->(device:Resource) joins a drawn item to the root
node of its
  datasheet graph. Several tags can share one datasheet graph (the same product). An item
without
  DESCRIBED_BY has no datasheet (e.g. a controller function).
- Every datasheet node has the property document_id (one value per datasheet) and a uri
that
  starts with 'urn:plantdocs:<document_id>:'. Parts: (part)-[:lis__partOf]->(device).
- For the facts of a tag: MATCH (p:PlantItem {tag: 'X'})-[:DESCRIBED_BY]->(dev)
  MATCH (d) WHERE d.document_id = dev.document_id AND (d:plm__PCA_100029511 OR
  d:plm__PCA_100029508 OR d:plm__PCA_100029510) ... and filter d by the words of the
question.
The datasheet graphs follow this model:
Ask the vendor datasheet of the item with this tag (pump, valve, transmitter, heat
exchanger, ...) one question about its specification, e.g. a pressure or temperature
limit, a material, a size or an option; returns the datasheet's answer with its PDF pages.
Use it for every property the drawing does not show; first find the tag on the drawing.
```

A sémakiegészítés ezután a plantdocs A2 `qa-cypher` promptjának GRAPH MODEL és RULES részét és a tag-ek dokumentumonkénti listáját tartalmazza.

D. A két forrásfolyamat részletei

Ez a függelék a 4. fejezetben összefoglalt két forrásfolyamatot írja le olyan részletességgel, amennyi a gráfok szerkezetének megértéséhez kell. A két folyamat kódját nem módosítottuk; amit leírunk, az a két egyéni munka terveiből és kódjából származik.

A plantgraph üzemgenerátora

Benedek rendszerében a generátor egy teljes üzemet épít, a lapokra bontó (splitter) lapokra vágja, a feloldó (resolver) a lapokból megoldókulcs nélkül visszaépíti az üzemet, végül a betöltő Neo4j-be tölti. Mivel a bontást a rendszer maga végzi, pontosan tudható, melyik lapközi csatlakozó melyikkel alkot párt, és ehhez mérhető a feloldó.

A generátor minden véletlen döntést egyetlen seedelt véletlenszám-generátoron át hoz meg, ezért ugyanaz a seed ugyanazt az üzemet adja; a kimenet valódi pyDEXPI-modell. Az üzem egységekből (unit) áll. Egy egységen belül a berendezéseket sorban veszi fel, és mindegyik új berendezés véletlenszerűen választ szülőt a már meglévők közül, amelytől anyagáram fut hozzá (véletlen rekurzív fa); így elágazó, nem lánc alakú topológia jön létre. Ezt három, valószínűséggel vezérelt kiegészítés teszi valószerűbbé: egy későbbi berendezésből visszaforgató ág futhat egy korábbiba, minden új egység egy korábbi egység egyik berendezéséből kap bemenő áramot, és egy korábbi egységből kereszt-kapcsolat is futhat egy későbbi egység belsejébe.

Az áramokra szelepek kerülnek, a berendezésekhez pedig adott valószínűséggel szabályozókör tartozik: mérő (távadó), szabályozó, működtető és a szabályozott szelep. A tag-ek a szokásos betűket követik: a kör változója F, L, P vagy T, a távadó `FT-...`, a szabályozó `FIC-...`, a szabályozott szelep `FV-....` A generátor terve kifejezetten elveti a véletlen címkéjű véletlen gráfot: a berendezések az áramok végén állnak, a szelepek az áramokon ülnek, a szabályozókörök a berendezésekhez tartoznak, és csak ettől lesz értelme egy „mi táplálja ezt a szivattyút?" kérdésnek. Benedek két generátort készített; a mi példaüzemeink az egyszerűbb, első generátorral készültek, mert a valósághűbb folyamatmintákat (oszlop, reaktor, biztonsági szelep) adó későbbi generátor berendezésosztályai nem szűkíthetők az adatlappal rendelkező osztályokra.

A generátor minden mezőt megjelöl aszerint, hogy látható-e a vizsgált rendszer számára: látható a tag, az osztály, a vezetékszám és a csatlakozó felirata, rejtett a belső azonosító és az áram fajtája, mert ilyen jelölés egy valódi rajzon nincs. Ha egy több lapon felrajzolt berendezés minden előfordulása ugyanazt a belső azonosítót viselné, a feloldónak nem kellene semmit kitalálnia, ezért minden előfordulás átlátszatlan azonosítót kap.

A plantgraph lapokra bontója és a lapközi csatlakozók

A lapokra bontó egy teljes üzemgráfot vág lapokra egy lapkeret szerint, és minden elvágott élre egy lapközi csatlakozópárt illeszt: a forrás lapján egy kimenő, a cél lapján egy bejövő csatlakozót, amelyek felirata megnevezi a partner lapot és csatlakozót. A bontás stratégiája egy közös felület mögött cserélhető: az alapértelmezett a folyamat irányát követi, és a lapot a keretéig tölti; más stratégiák egységek szerint vagy közösségkereséssel vágnak, illetve a segédközegeket külön lapokra teszik; egy véletlen stratégia pedig kontrollként szolgál. A bontó csak lapgráfokat állít elő; a rajzexport külön komponens.

A laphatár kétféle kapcsolatot vághat el, és a bontó mindkettőt előállítja (lásd az alábbi táblázatot). A megoldókulcsot (csatlakozópárok, azonossági csoportok) külön fájlba írja, amely a vizsgált adatbázisba nem kerül be; a lapokból és a megoldókulcsból pontosan az eredeti gráfnak kell visszaállnia.

10. táblázat: A két lapközi mechanizmus

	Lapközi csatlakozó	**Tag-azonosság**
Mi vágódik el	egy cső: egy él két részre	egy elem: egy csúcs két lapon
Visszaállítás	az elvágott él újraösszekötése	a duplikált csúcsok összevonása
Jel a rajzon	a csatlakozó felirata a partner lappal	az elem tag-je
Ha elmarad	a gráf szétesik, a hiba látható	a gráf épnek látszik, a válaszok hiányosak

A példaüzem 4 lapja között 4 lapközi csatlakozópár van; a rajzexportban mind a 8 lapközi jel összekötött.

A plantgraph feloldója

A feloldó bemenete csak a lapok sorozata; a megoldókulcsot nem látja. A csatlakozók feliratából két csatlakozó akkor lesz pár, ha kölcsönösen egymásra hivatkoznak (mindkettő

a másik lapját és csatlakozószámát nevezi meg); a maradékot a vezetékszám és a közegkód szerint csoportosítja, és csak akkor párosít, ha egy csoportban pontosan egy kimenő és egy bejövő csatlakozó van. Minden más feloldatlan marad, megnevezett okkal; a feloldó nem találgat.

Az azonossági csoportokat a (tag, osztály) kulcs adja: ha egy ilyen kulcs két vagy több lapon fordul elő, az előfordulások egy elemet jelentenek. Alapértelmezésben csak berendezéseket von össze, és csak pontos egyezéssel. Végül az összefésülés minden csoportot a legtöbb tulajdonságot hordozó előfordulásával helyettesít, a csatlakozópárokon át újraköti az elvágott éleket, a párosítatlan csatlakozókat pedig meghagyja, hogy a hiány a kapuellenőrzésben látsszon. A legfőbb kapu az egyezés: a feloldott üzemnek az eredeti üzem látható nézetével kell megegyeznie; egy átnevezési próba pedig azt ellenőrzi, hogy az eredmény nem függ az átlátszatlan azonosítóktól. A példaüzemen a visszaépítési ellenőrzés hibátlan volt.

Az adatlapok PDF-jei

A hét adatlap valódi, a gyártók által nyilvánosan közzétett PDF: Gorman-Rupp VGH (0700286), Alfa Laval M3, Fisher ET (D100022X012), GF Type 165, Flowserve Worcester F51/F52, Swagelok MS-01-176 és Rosemount 3051 (00813-0100-4001). A négy általunk feldolgozott adatlap PDF-je a jelen munka kódtárában, a másik három Szabó Zsombor egyéni munkájának kódtárában található. A forrásoldal-hivatkozások ezeknek a PDF-eknek az oldalszámaira mutatnak.

A plantdocs folyamatának lépései

A plantdocs egy gyártói adatlap PDF-jéből tíz számozott lépésben jut el az ontológia szerinti RDF-gráfig. A lépések felváltva programok és nyelvi modellek; minden nyelvimodell-lépés kimenete csak akkor mehet tovább, ha átmegy egy programmal ellenőrzött formai és tartalmi feltételrendszeren. A programlépéseket egy visszamenőleges teszt újrafuttatja és összeveti a tárolt kimenettel. A lépéseket az alábbi táblázat foglalja össze.

11. táblázat: A plantdocs folyamatának lépései

#	Ki végzi	Mit csinál
1	Azure Document Intelligence	elrendezés-elemzés: oldalelrendezés, táblázatok és ábrák felismerése
2	nyelvi modell	rekordok: paraméter, érték, egység, a táblázatbeli útvonal
3	program, nyelvi modell, program	ábrák kivágása és osztályozása, a betű- és számjelek feloldása
4	program, nyelvi modell, program	diagramoldalak, leolvasás, vektoros ellenőrzés
5	program	a rekordok ellenőrzése a PDF szövegével
6	nyelvi modell, program	célformátum: tények, feltételek, lábjegyzetek, diagramok
7	program	az ábrákhoz tartozó lábjegyzetek
8	program, nyelvi modell, program	forrásleltár és forrásleírások, a végső célformátum

#	Ki végzi	Mit csinál
9	program, nyelvi modell, program	szókészlet: illesztés, hálózati ellenőrzés, szókészlet-térkép
10	program	RDF: IDO és POSC Caesar adatlap-ontológia

Az **1.** lépés az Azure Document Intelligence elrendezés-elemzése (bekezdések, táblázatok, ábrák); a plantdocs ezt adott eszköznek tekinti, nem kutatási hozzájárulásnak. A **2.** lépésben egy nyelvi modell rekordokat ír: paraméter, érték, egység és a táblázatbeli útvonal (táblázatcím, sor- és oszlopfejlécek) a lábjegyzetjelekkel. Ezek még lapos szövegek.

A **3.** lépés az ábrákat kivágja és osztályozza (fénykép, metszeti rajz, méretrajz, diagram), és a rajzokon a betű- és számjeleket alkatrésznevekhez vagy táblázatfejlécekhez köti; amit nem tud feloldani, azt feloldatlanként megtartja. A **4.** lépés a diagramokat olvassa le: egy szelepdiagram üzemi tartományok sokszögeiből, egy görbediagram pontsorozatokból áll; ahol lehet, a leolvasást a PDF vektoros vonalaival ellenőrzi, és a diagram fölötti címsorok feltételeit a diagramhoz rendeli. Az **5.** lépés minden rekord számát megkeresi a hivatkozott PDF-oldal szövegében.

A **6.** lépés hozza létre a célformátumot (lásd alább). Ehhez a plantdocs hibrid módszert használ: a program a rekordok útvonalát szegmensekre bontja, egy nyelvi modell táblázatonként egyszer megadja, melyik szegmens az alany, melyik a feltétel és melyik a tulajdonság (táblázatséma), és egy determinisztikus lépés ezt minden rekordra alkalmazza. Ez olcsóbb, mint rekordonként kérdezni, és egy táblázaton belül következetes. A **7.** lépés az ábrák lábjegyzeteit köti be, a **8.** lépés minden forrásról rövid leírást írat; ezek a lekérdezésnél a szakaszkatalógus sorai lesznek.

A **9.** lépés a szókészlet-illesztés. Egy nyelvi modell minden eszköztulajdonsághoz és feltétel-dimenzióhoz szabványos fogalmat keres, a program minden javasolt azonosítót újra lekérdez a hálózaton, és csak az ellenőrzött illesztés kerül a szókészlet-térképbe. A sorrend rögzített: előbb a POSC Caesar könyvtár pontos vagy tágabb fogalma, ha nincs, a CFIHOS-é; modell- vagy változatkódnál a könyvtár azonosítási osztálya; különben az IDO legáltalánosabb tulajdonság-osztálya (`Quality`) mint gyűjtő, ahol a pontos jelentést a nyomtatott címke hordozza. A gyűjtőbe kerülő tények aránya az illesztés minőségének jelzője. Az illesztés lefedettségét Szabó Zsombor egyéni munkájában egy előzetes mérés vizsgálta: négy dokumentum eszköztulajdonságai és feltétel-dimenziói közül 88/133-hoz adott a POSC Caesar könyvtár pontos vagy tágabb fogalmat, a CFIHOS-szal együtt 101/133-hoz (előzetes, kézi ellenőrzésre váró eredmény). Végül a **10.** lépés program: a végső célformátumot és a szókészlet-térképet RDF-gráffá alakítja, és jelentést ír a tények szintek szerinti eloszlásáról.

Az általunk feldolgozott négy adatlapon minden nyelvimodell-kimenet elsőre átment a plantdocs ellenőrzésein.

A plantdocs célformátuma

A célformátum a kinyerés és a gráfépítés között álló, ontológiafüggetlen rekordformátum. Öt elvre épül:

1. Egy rekord egy tény, nem egy nyomtatott cella: ugyanannak a ténynek a két mértékegységben nyomtatott cellája egy rekord két nyomtatott értékkel.
2. A feltételek elsőrendű elemek, egy lista, nem egy név része. Egy tény a feltételei nélkül nem hiányos, hanem hibás.
3. Semmit nem javít csendben: a nyomtatott forma megmarad, a gyanús elírás jelzőt kap, és nem íródik felül.
4. Minden visszakövethető a forrásrekordokig és a PDF-oldalig.
5. A mutató nem érték: a kereszthivatkozás és a lábjegyzetjel típusos, hogy a gráfban él lehessen belőle.

Egy tényrekord vázlata a következő (a mezőnevek a plantdocs formátumáé, az értékek helyén leírás áll):

```
{
  "subject":    {"label": "<az alany, ahogy nyomtatva van>", "kind": "<alkatrész
fajtája>"},
  "property":   {"label": "<a tulajdonság nyomtatott neve>", "key": "<rövid kulcs>"},
  "value_type": "quantity | code | text | reference | not_limiting | expression",
  "values":     [{"value": "<szám>", "unit": "<egység>", "printed": "<ahogy nyomtatva
van>"}],
  "conditions": [{"dimension": "<feltétel-dimenzió>", "any_of": ["<megengedett
értékek>"]}],
  "footnotes":  ["<lábjegyzetrekordok azonosítói>"],
  "flags":      ["<például gyanús elírás, a mértékegység-pár nem egyezik>"],
  "source":     {"document": "<dokumentum>", "page": "<PDF-oldal>", "table": "<táblázat>"}
}
```

Az értéktípus megkülönbözteti a mennyiséget (erre illeszkedik a QUDT), a kódot (anyagjel, nyomásosztály), a szabad szöveget, a hivatkozást (egy másik táblázatra mutat, értéke nincs), a „nem korlátozó” jelölést (ami információ, nem hiány) és a képletet. A kimeneti fájl hat listát tartalmaz: tények, lábjegyzetek, ábrák, ábra–alkatrész kapcsolatok, diagramok és indoklással kihagyott rekordok; minden bemeneti rekord pontosan egy helyre kerül, semmi nem veszhet el csendben. A lábjegyzetben kimondott tények önálló tények, amelyek a lábjegyzetükre is visszamutatnak.

A plantdocs RDF-modellje

A plantdocs ontológiai döntése az ISO 15926 vonal: az IDO a felső ontológia, a POSC Caesar könyvtára az eszköz- és referenciaszótár, a mértékegység QUDT. A döntés mellett szólt, hogy szabványos és nyílt, és hogy a DEXPI is innen indul; ellene a tanulási görbe, és hogy a könyvtár eszközmodulja nem ad teljes tulajdonságkatalógust. A legfontosabb megkötés az, hogy saját, kézzel írt szókészlet nem lehet: minden osztály és tulajdonság szabványos azonosító, ami pedig a dokumentumból jön (címke, érték, feltétel), az adat, nem szókészlet. Ha nincs pontos szabványos fogalom, a legközelebbi tágabb kerül a gráfba, a pontos jelentést a dokumentum szövege hordozza. A rétegeket az alábbi táblázat mutatja.

12. táblázat: Az adatlapgráf rétegei

Réteg	Szabvány	Mire
Nyelvtan	IDO, `lis:` előtag	fizikai tárgy, rész–egész, információs tárgy, érték és egység
Adatlap-szerkezet	POSC Caesar adatlap-ontológia	adatlap, bejegyzés (datum), alany, attribútum, eset, szakasz

Réteg	Szabvány	Mire
Eszközosztály	POSC Caesar könyvtár	például szolenoidszelep, centrifugálszivattyú
Tulajdonság, feltétel-dimenzió	POSC Caesar könyvtár, ahol nincs: CFIHOS	például névleges átmérő
Mértékegység	POSC Caesar egységmodul és QUDT	azonosító, illetve mennyiségfajta és átváltás
Egyéb	Dublin Core, SKOS, schema.org	leírás, jelölés, oldalszám

A leképezés a célformátumból a gráfba a következő. Az eszköz egy `lis:PhysicalObject` csúcs a POSC Caesar eszközosztályával; az alkatrészek szintén fizikai tárgyak, `lis:partOf` kapcsolattal az eszközre. Egy mennyiségi tény mennyiségi adatpont (Quantity Datum) a számmal, a mértékegységgel és tartomány esetén alsó és felső határral; egy szöveges vagy kód jellegű tény attribútum-adatpont (Attribute Datum); ha az érték maga szabványos osztály (például egy anyag), osztályozó adatpont (Classification Datum). Minden adatpont hordozza a nyomtatott címkét és értéket, a PDF-oldalt, az alanyát és, ha van, a szabványos attribútumát.

A feltételek együttese egy esetcsúcs (Case), amelyet a POSC Caesar úgy határoz meg, mint egy helyzetet, amelyen belül bizonyos attribútumértékek érvényesek. A tény egy kapcsolattal az esetre mutat, az eset címkéje felsorolja a feltételeket, és minden egyes feltétel maga is adatpont, amely ugyanerre az esetre mutat, a dimenzió szabványos attribútumával. A forrás (táblázat, ábra, diagram, szövegrész) szakaszcsúcs (Section Datum) a címmel, a 8. lépés leírásával és az oldalszámmal; a lábjegyzet külön adatpont, amelyre a tény kapcsolattal mutat. A diagramok pontjai pontonként egy-egy esetbe kerülnek.

Miért eset, és nem valami más? A feltételes tény n-áris, ezért kell egy csúcs, amely a feltételeket összefogja (2. fejezet). A plantdocs megvizsgált egy saját tény-csúcsot is, de a POSC Caesar adatlap-mintája ugyanezt szabványosan tudja, a saját szókészlet pedig ki volt zárva. Mérlegelte az IDO áram-mintáját is, amely az üzemi környezetet (nyomás, hőmérséklet, közeg) egy áramként modellezi, amelynek a szelep résztvevője; ez azonban egy üzemi helyre vonatkozó előírást ír le, nem egy gyártói adatlap feltételeit. A döntés végül az egységesség volt: minden feltétel-kombináció eset. Ennek ára, hogy az „eset” szót a szabvány elsősorban feltételezett helyzetekre használja, a plantdocs pedig termékváltozatokra és üzemi feltételekre is.

E. A módszerek részletei

Ez a függelék az 5. fejezetben leírt módszerek egyes lépéseit részletezi.

Az ágens primitívjei

Benedek ágense hat általános, egymással komponálható primitívet hív. A primitívek a feloldó kimenetéből a memóriában épített elemgráfon futnak, amelyben a lapközi csatlakozók és a többszörösen felrajzolt elemek már össze vannak vonva; ez a gráf a betöltött üzemi réteggel egyezik. Az ágens tehát nem látja a laphatárokat, csak az üzemet.

13. táblázat: Az üzemi primitívek

Primitív	Mit ad vissza
`find`	a szűrőnek megfelelő elemek (osztálycímke, osztály, egység, lap, tag, fajta szerint)
`neighbours`	egy elem közvetlen szomszédai egy relációcsoport mentén
`traverse`	bejárás egy kiinduló elemtől, megállási és szűkítő feltétellel
`path`	út két elem között
`filter`	egy korábbi eredmény szűkítése a közös szűrőnyelvvel
`aggregate`	egy korábbi eredmény csoportosítása vagy összesítése

A primitívek eredményei hivatkozást (handle) kapnak, így egy korábbi lépés eredménye a következő bemenete lehet. A szűrőnyelv mind a hat primitívnél ugyanaz, és ugyanazokat a címkeneveket használja, mint a Cypher-szint sémaszövege; a hangolt rendszer ezt az elem fajtájával és a vonalnézet relációcsoportjával bővítette. A tervezési döntés lényege, hogy az eszközök száma a gráfsémából következik, nem a kérdéstípusokból: a hat primitív a séma szókincsén értelmezett szokásos gráfműveletek (kiválasztás, kiterjesztés, lezárás, legrövidebb út, szűkítés, csoportosítás), ezért egy új kérdéstípus egy új kompozíció, nem egy új eszköz. Az egyszerű JSON-akciók minden modellen ugyanúgy működnek, és a gyorsítótár változtatás nélkül vissza tudja játszani őket.

Az adatlapoldali A2 lépései

A plantdocs A2-módszere egy eszköz egy kérdésére a következő lépésekben válaszol.

1. **Kontextus.** A rendszerüzenet a plantdocs gráfmodelljének leírását (a 4. fejezetben bemutatott adatpont-, eset-, szakasz- és lábjegyzetcsúcsokat az előtagos neveikkel), az eszköz gráfjának leltárát (az eszköz és alkatrészei, a leggyakoribb adatpontcímkék, a források oldallal) és a forrásszakaszok katalógusát tartalmazza; a katalógus soronként egy táblázat, ábra, diagram vagy szövegrész a címével, típusával, oldalával, sorainak számával és a 8. lépésben írt leírásával.
2. **Lekérdezésírás.** Egy LLM alacsony gondolkodási szinttel egyetlen csak olvasó Cypher-lekérdezést ír, és mellette legfeljebb három forrásszakaszt választ, amelynek a teljes tartalmára a válasznak valószínűleg szüksége lesz. A lekérdezési szabályok széles keresést kérnek: a kérdés egy szava a címkében, a nyomtatott értékben, az eset címkéjében (a feltételekben) vagy egy forrásszakasz címében és leírásában is lehet; a lekérdezésnek mindig vissza kell adnia a nyomtatott értéket, a címkét, az esetet, a lábjegyzetet, a forrást és az oldalt.
3. **Futtatás és javítás.** Ha a lekérdezés hibát ad, üres, vagy túl sok sort ad vissza, a modell egy javítókörben megkapja a problémát („javítsd vagy bővítsd”, illetve „szűkítsd a legjellemzőbb szavakra”). Ha a választott szakaszokból jött sor, javítókör nincs; ha a javítás üres eredményt ad, az első, nem üres eredmény marad.
4. **Teljes szakaszok.** A kiválasztott szakaszok minden sora a lekérdezés sorai mellé kerül; ez azt az esetet fedi le, amikor a táblázat fejlécei nem a kérdés szavait használják.
5. **Vektoros keresés.** A lekérdezésírással párhuzamosan a kérdés beágyazásvektorához legközelebbi 30 adatpont kerül elő a gráf vektorindexéből, mindegyik az esetével, a lábjegyzetével, az oldalával és a forrásszakaszával. Mivel párhuzamosan fut, a késleltetéshez nem ad hozzá.

6. **Válaszírás.** A válaszíró csak a megtalált tényekből fogalmaz: előbb egy-két mondatban a közvetlen választ, utána az értékeket az egységükkel és azokkal a feltételekkel, amelyek mellett érvényesek, oldalhivatkozással. A kérdést abban az értelemben kell olvasnia, ahogy az adatlap használja a szavait; egy szakkifejezést néhány szóval megmagyarázhat, de új tényt, értéket vagy eljárást nem tehet hozzá; ha a kérdés egy része nincs a tények között, ezt kimondja. A forrásoldalakat a program a felhasznált sorokból gyűjti össze.

Az M1 sémakiegészítésének felépítése

A sémakiegészítés első része az összekötést írja le (a `DESCRIBED_BY` nélküli elemnek, például egy szabályozófunkciónak nincs adatlapja; minden adatlapcsúcsnak van `document_id` tulajdonsága; egy tag tényei a fenti mintával érhetők el). Ezt követi a plantdocs A2-promptjának gráfmodell-része szó szerint, a dokumentumok és tag-jeik listája, végül a plantdocs lekérdezési szabályai, amelyek a második fejlesztési iterációban kerültek be. A sémakiegészítés elejét a Függelék C mutatja.

Az M2 router protokollja

A rendszerüzenet leírja a két eszközt, felsorolja az adatlappal rendelkező tag-eket, megadja a három megengedett JSON-alakot (rajzeszköz-hívás, adatlap-hívás, végső válasz a „nincs benne” jelzővel) és a válaszalakot. A router minden lépésben pontosan egy JSON-objektumot ír; a program végrehajtja a hívást, és az eredményt egy új üzenetben („Tool result:”) adja vissza. Érvénytelen akcióra a router javító üzenetet kap; ha a 16 lépéses keret elfogy, a router egy utolsó hívásban köteles végső választ adni a már összegyűjtött eredményekből (a prototípusban ilyenkor a kérdés lépéskorlát-eredménnyel zárult). Szolgáltatói hibánál (például hálózati hibánál) a hívás egyszer újrapróbálkozik, ahogy Benedek kaszkádja is; ha ez is hibás, a kérdés szolgáltatói hibával zárul, és rossznak számít. A router alacsony gondolkodási szinttel fut. A rendszerüzenet teljes szövegét a Függelék B tartalmazza.

Útvonalak a tesztüzemeken

Az M1 GLM-mel a 100 lapos tesztüzemen a 60 kérdésből 31-et a Cypher-szinten zárt le, 9-hez az ágens is kellett adatlap-hívás nélkül, 20-hoz az ágens és legalább egy adatlap-lekérdezés; Lunával 34, 5 és 21. Az 1000 lapos üzemen ugyanez GLM-mel 31, 11 és 18, Lunával 39, 6 és 15. Az M2 nagyjából harmadolta a kérdéseket a csak rajzi, a csak adatlapi és a mindkét eszközt hívó útvonal között (a 100 lapos üzemen GLM-mel 19, 17 és 24, Lunával 21, 18 és 21; az 1000 lapos üzemen 25, 15 és 20, illetve 21, 19 és 20), ami a kérdéscsaládok megoszlásával egyezik.

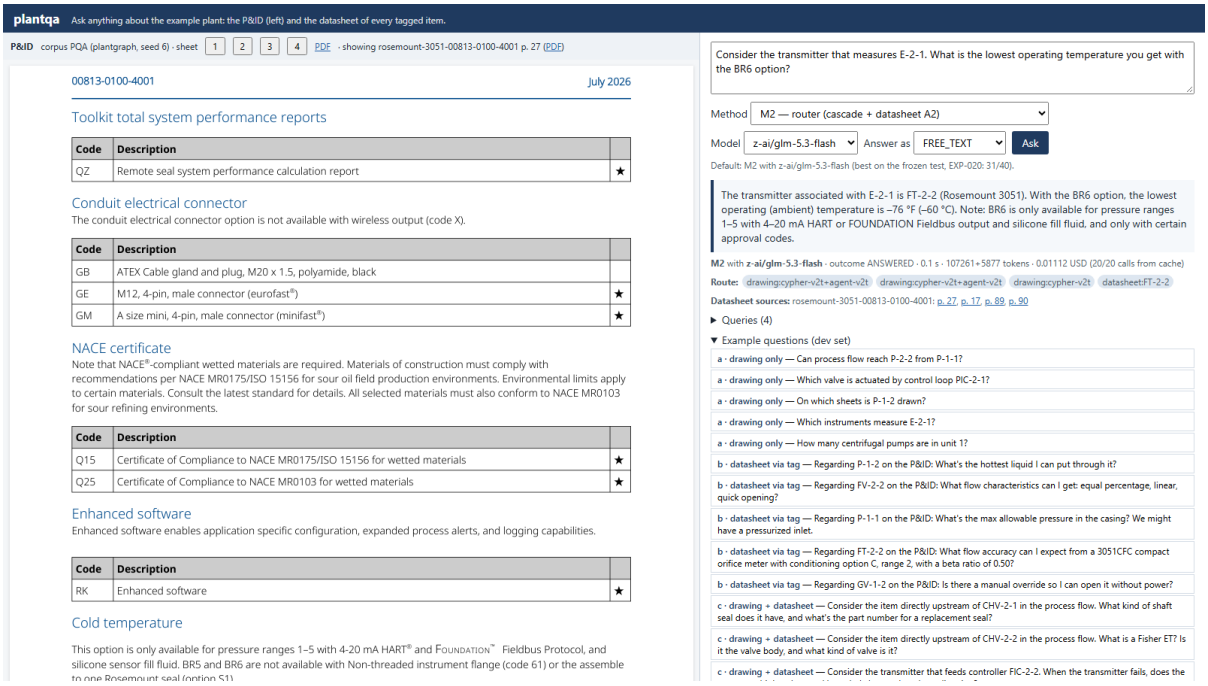
F. További demó-képernyőképek

Két kérdés végigkövetve

Összetett kérdés a routerrel

Az első kérdés az összetett (c) családból való: „Consider the transmitter that measures E-2-1. What is the lowest operating temperature you get with the BR6 option?” A kérdés a hőcserélőt nevezi meg, a választ viszont a hőcserélőt mérő távadó adatlapja tartalmazza, és ott is csak egy rendelési opcióhoz kötve. A kérdés tehát egy topológiai lépést (melyik műszer

méri az E-2-1-et?) és egy feltételes adatlapi tényt (a legalacsonyabb hőmérséklet a BR6 opcióval) kapcsol össze.

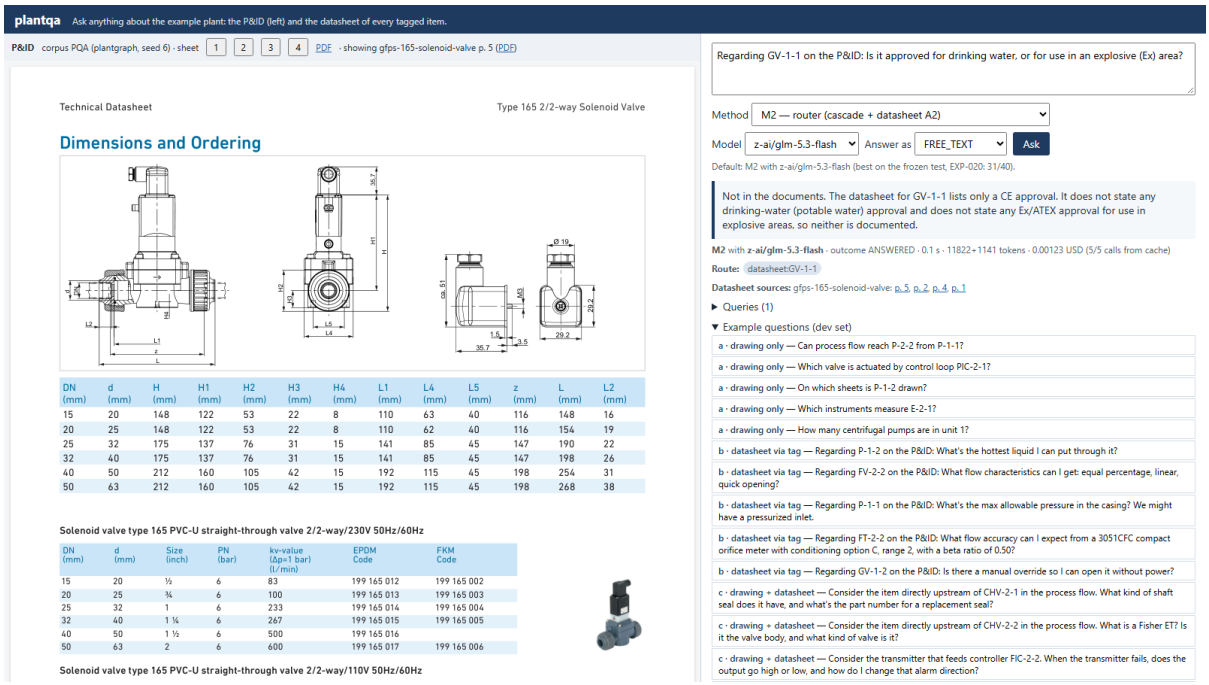


5. ábra: Összetett kérdés az M2-vel: a router előbb a rajzeszközt, majd a távadó adatlapját kérdezi; a bal oldalon a válasz egyik forrásoldala a Rosemount 3051 adatlapjából.

Az M2 útvonala: a router háromszor kérdezte a rajzeszközt, ebből kétszer a kaszkád a Cypher-szintről az ágensig jutott, egyszer a Cypher-szinten zárult; ezután az FT-2-2 adatlapját kérdezte. A válasz így kezdődik: „The transmitter associated with E-2-1 is FT-2-2 (Rosemount 3051).” Ezután megadja a BR6 opcióval elérhető legalacsonyabb környezeti hőmérsékletet Fahrenheit- és Celsius-fokban, és hozzáteszi, hogy a BR6 csak bizonyos nyomástartományokkal, kimenettel és töltőfolyadékkal rendelhető. Ez a plantdocs válaszírójának szabálya: egy érték a feltételei nélkül hibás lehet. A bal oldalon az adatlap azon oldala látszik, amely a hideg környezetre vonatkozó opció feltételeit írja le. A router tehát a tag-et a rajzból szerzi meg, és az adatlapi kérdést már a tag-re teszi fel, ahogy a rendszerüzenete előírja; a rajzeszköz ismételt hívása viszont a késleltetést növeli.

Megválaszolhatatlan kérdés és a részletes „nincs benne”

A második kérdés a (d) családból való, amelynek kérdéseire a dokumentumok nem adnak választ: „Regarding GV-1-1 on the P&ID: Is it approved for drinking water, or for use in an explosive (Ex) area?” A kérdés tag-et nevez meg, ezért az M2 routere a rajzeszköz nélkül, közvetlenül a GV-1-1 adatlapját kérdezte (csak az adatlap-eszközt hívta, a GV-1-1 tag-re).



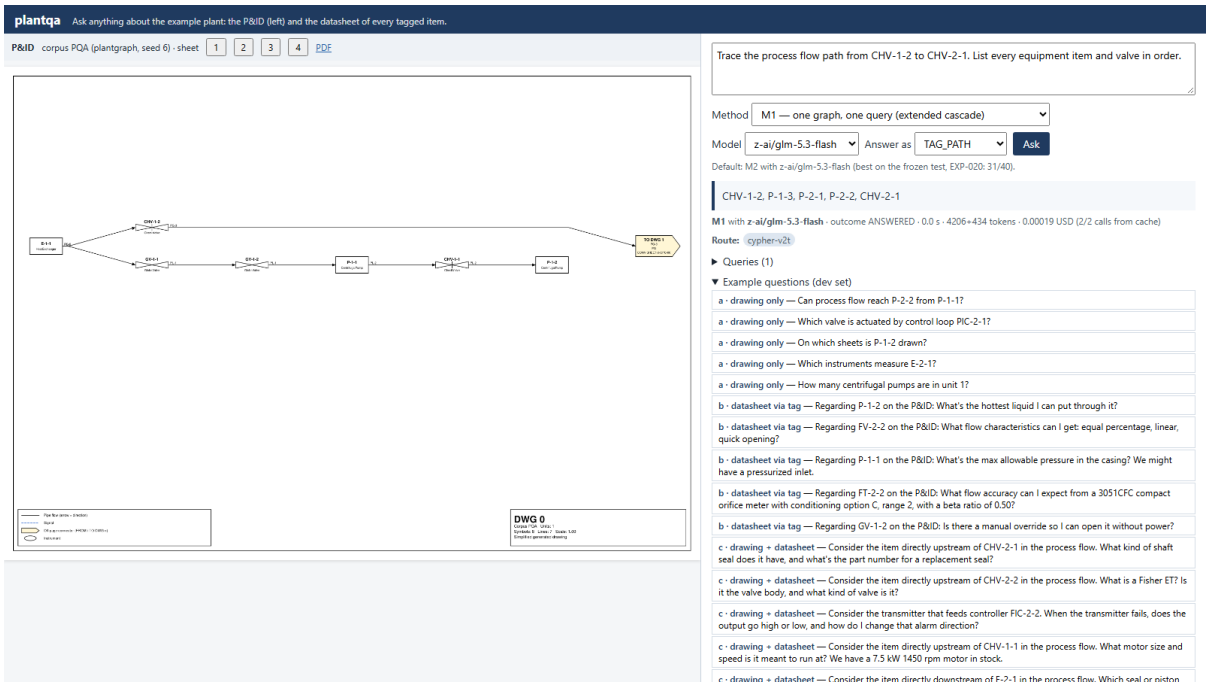
6. ábra: Megválaszolhatatlan kérdés az M2-vel: a router közvetlenül az adatlapot kérdezi, a válasz „nincs benne”, de megnevezi, mit tartalmaz az adatlap; a bal oldalon a GF 165 adatlap egyik forrásoldala.

A válasz „nincs benne” jelzésű, de nem áll meg ennyinél: „Not in the documents. The datasheet for GV-1-1 lists only a CE approval. It does not state any drinking-water (potable water) approval and does not state any Ex/ATEX approval…”. A felhasználó így azt is megtudja, mit tartalmaz az adatlap a tanúsításokról, és ellenőrizheti a forrásoldalon. Ez a megjelenítés egy hibajavítás eredménye, amelyet a demó egy korábbi próbája derített ki: a válasz megjelenítése és a bírálónak exportált jelölt szöveg a „nincs benne” jelzésnél eredetileg csak a jelzést mutatta, és elnyelte a mellette írt magyarázatot. A hibát kijavítottuk, az érintett dev-tételeket a javított szöveggel újrabíráltattuk, és a végső teszt exportja már a javított kóddal készült. A demó tehát nemcsak bemutató, hanem ellenőrző eszköz is volt: olyan hibát talált meg, amely a pontozást is érintette.

A 7. fejezetben bemutatott demóban hat kérdést követünk végig: kettőt részletesen, kettőt képernyőképpel és kettőt képernyőkép nélkül.

Rajzi útvonal egyetlen lekérdezéssel (M1)

A demó próbájának egy további kérdése a rajzi (a) családból való, rendezett út válaszalakkal: „Trace the process flow path from CHV-1-2 to CHV-2-1. List every equipment item and valve in order.” Az M1-gyel feltéve a kaszkád a Cypher-szinten zárult, egyetlen lekérdezéssel, és a válasz egy tagsorozat: „CHV-1-2, P-1-3, P-2-1, P-2-2, CHV-2-1”.

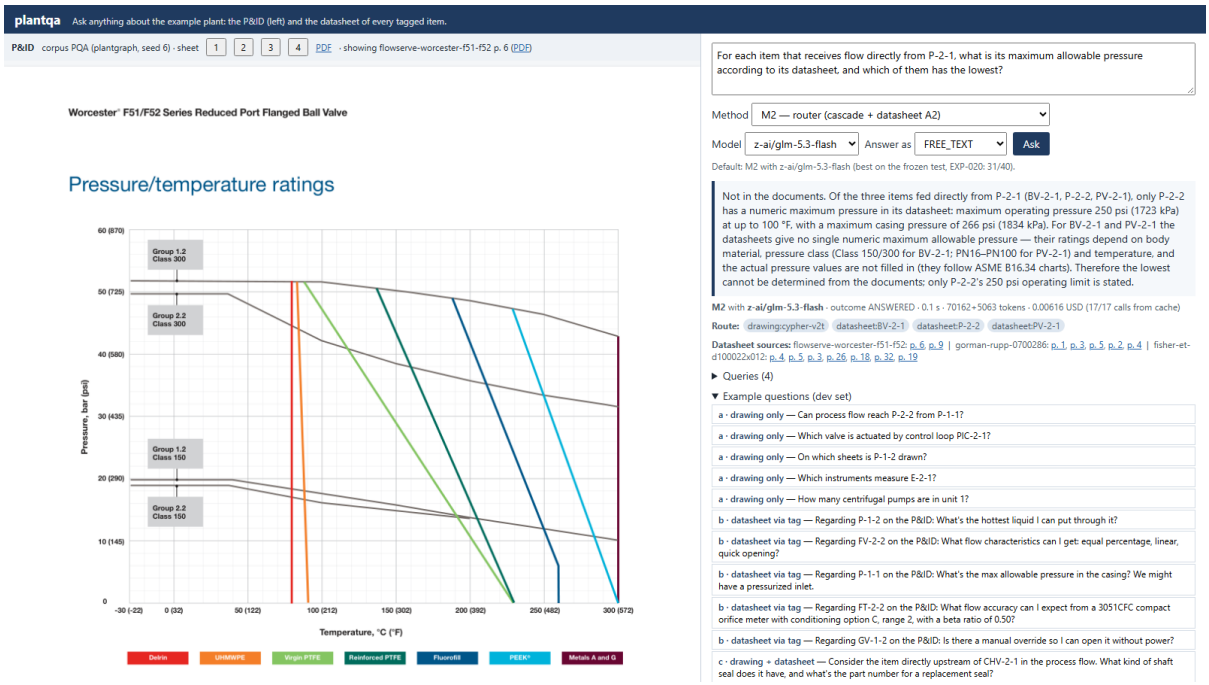


7. ábra: Rajzi útvonal az M1-gyel: a Cypher-szint egyetlen lekérdezéssel adja a rendezett utat; a bal oldalon a P&ID egyik lapja, a jobb szélén a lapközi csatlakozóval.

A bal oldalon a rajz egyik lapja látszik. A CHV-1-2 szelepből induló vezeték a lap jobb szélén egy lapközi csatlakozóban végződik, vagyis az út egy másik lapon folytatódik. A rajzon ezt az olvasónak kell követnie; az üzemi rétegben viszont a laphatár átlépése egyetlen él (4. fejezet), ezért a Cypher-szint egyetlen lekérdezéssel megtalálta a teljes utat, és a kaszkád az elfogadási szabálya szerint (válaszolt, nem tartózkodott, van sor, nem csonka) elfogadta.

Több adatlapos kérdés (M2)

Egy másik kérdés a külön mért, több adatlapos (e) családból való: „For each item that receives flow directly from P-2-1, what is its maximum allowable pressure according to its datasheet, and which of them has the lowest?” A kérdés egy halmazt kér a rajzból (mi kap közvetlenül áramot a P-2-1-ből?), majd a halmaz minden elemére egy adatlapi értéket, végül egy összevetést.



8. ábra: Több adatlapos kérdés az M2-vel: egy rajzi hívás után három adatlapi hívás; a bal oldalon a Worcester gömbcsap adatlapjának nyomás–hőmérséklet diagramja.

Az M2 útvonala: egy rajzeszköz-hívás, amely a Cypher-szinten zárult, majd három adatlap-hívás a BV-2-1, a P-2-2 és a PV-2-1 tag-re, vagyis egy gömbcsap, egy szivattyú és egy szabályozószelep adatlapjára. A válasz „nincs benne” jelzésű részválasz: „Of the three items fed directly from P-2-1 (BV-2-1, P-2-2, PV-2-1), only P-2-2 has a numeric maximum pressure in its datasheet…” A folytatás szerint a másik két elem adatlapja nem ad egyetlen számszerű határt, mert a névleges nyomás a testanyagtól, a nyomásosztálytól és a hőmérséklettől függ, ezért a legkisebb érték a dokumentumokból nem dönthető el. A bal oldali nyomás–hőmérséklet diagram éppen ezt a függést mutatja.

Ez a válasz az 5. fejezetben leírt válaszszabály működését mutatja: a módszer a meglévő részt adja, és megnevezi a hiányzót, ahelyett hogy egy értéket találna ki. Hogy a válasz helyes-e, azt a próba nem pontozta.

Két további kérdés képernyőkép nélkül

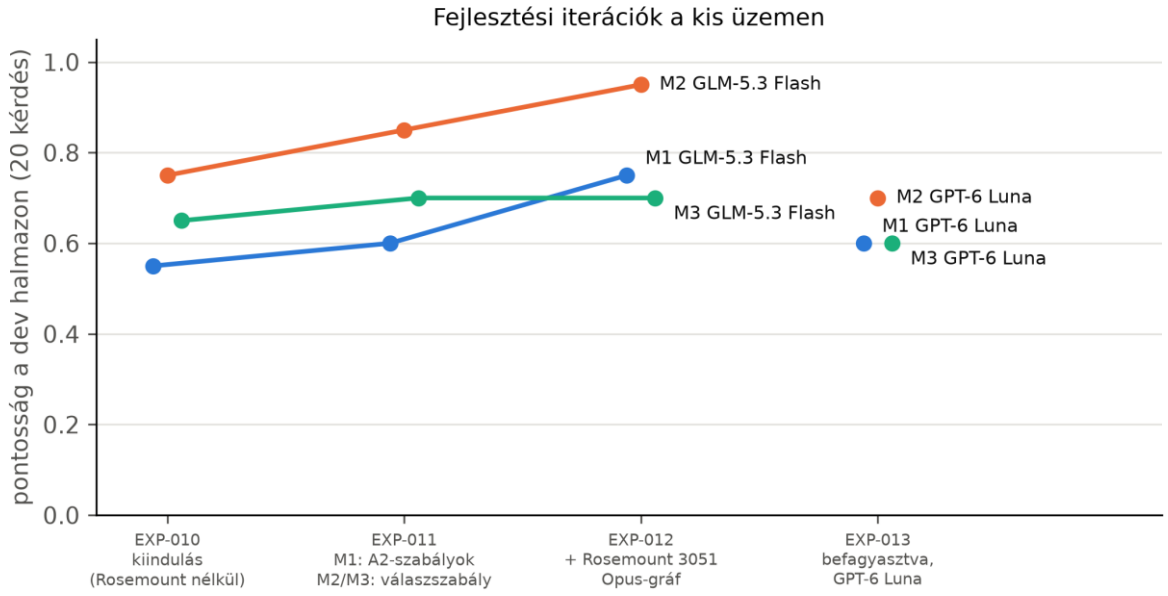
Az adatlapi (b) kérdésre („Regarding P-1-3 on the P&ID: What's the max allowable pressure in the casing? …”) az M1 a Cypher-szinten, egyetlen lekérdezéssel válaszolt; a rajzi (a) kérdésre („Which instruments measure E-2-1?”) az M3 egyetlen hívással az FT-2-2 tag-et adta.

G. A kis üzem mérései

Ez a szakasz az első prototípus mérését írja le: a módszerek első változatát a négylapos kis üzemen fejlesztettük és fagyasztottuk be, és ott mértük meg (változatlan számokkal). A közös rendszert ezután a 30 és 50 lapos fejlesztési üzemen hangoltuk (5. fejezet).

Fejlesztési iterációk

A dev halmazon (20 kérdés) négy iterációt mértünk (lásd az alábbi ábrát). Az első iterációban a Rosemount 3051 gráfja még nem készült el, ezért a nyomás- és áramlástávadóra vonatkozó adatlapi kérdések egyik gráfos módszerrel sem voltak megválaszolhatók. A második iterációban az M1 Cypher-szintje megkapta a plantdocs A2 lekérdezési szabályait (széles keresés, eset, lábjegyzet és oldal visszaadása), az M2 és az M3 pedig egy válaszszabályt (részválasznál a meglévő rész és a hiányzó rész megnevezése, a dokumentumon túli tanács tilalma); az M2 és az M3 hallucinációi ezzel eltűntek. A harmadik iterációban a Rosemount-gráf is bekerült: az M1 pontossága 0,60-ról 0,75-re, az M2-é 0,85-ről 0,95-re nőtt, az M3-é nem változott, mert nem használja a gráfot. A negyedik iterációban a befagyasztott módszereket GPT-6 Lunával mértük: M1 0,60, M2 0,70, M3 0,60.



9. ábra: A dev halmazon mért pontosság iterációnként.

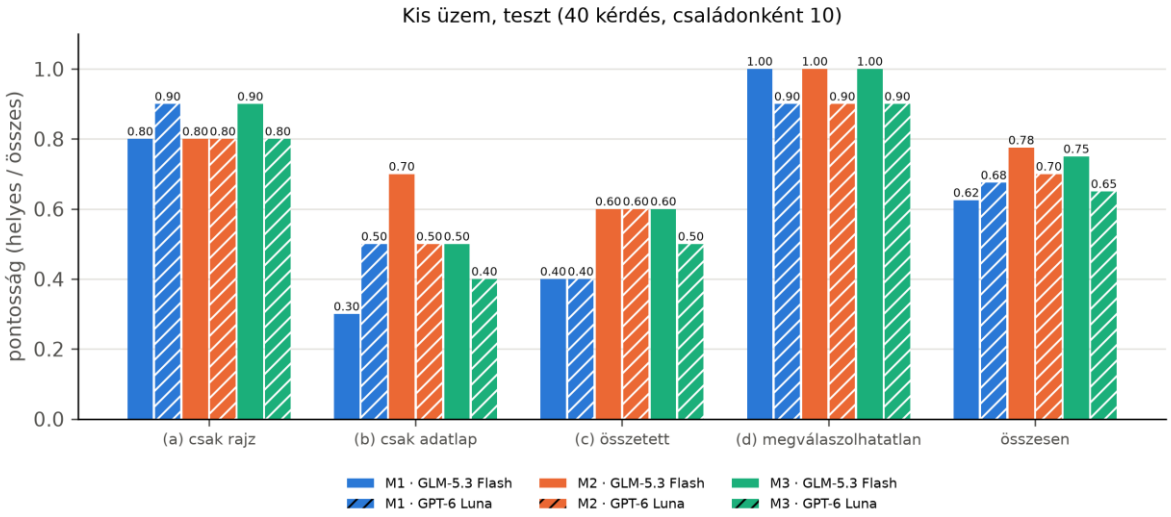
Végső mérés a teszthalmazon

A befagyasztott teszthalmaz (40 kérdés, családonként 10) mindegyik párosításon egyszer futott le. Az eredményt az alábbi táblázat és ábra mutatja.

14. táblázat: A végső teszt eredménye párosításonként és családonként

Módszer	Modell	Helyes	Részben	Rossz	Hallucinált	Pontosság [95% CI]	(a)	(b)	(c)	(d)
M1	GLM-5.3 Flash	25	9	6	0	0,625 [0,475–0,775]	8/10	3/10	4/10	10/10
M2	GLM-5.3 Flash	31	6	3	0	0,775 [0,625–0,900]	8/10	7/10	6/10	10/10
M3	GLM-5.3 Flash	30	1	9	0	0,750 [0,600–0,875]	9/10	5/10	6/10	10/10
M1	GPT-6 Luna	27	6	7	0	0,675 [0,525–0,825]	9/10	5/10	4/10	9/10

Módszer	Modell	Helyes	Részben	Rossz	Hallucinált	Pontosság [95% CI]	(a)	(b)	(c)	(d)
M2	GPT-6 Luna	28	7	5	0	0,700 [0,550–0,850]	8/10	5/10	6/10	9/10
M3	GPT-6 Luna	26	6	8	0	0,650 [0,500–0,800]	8/10	4/10	5/10	9/10



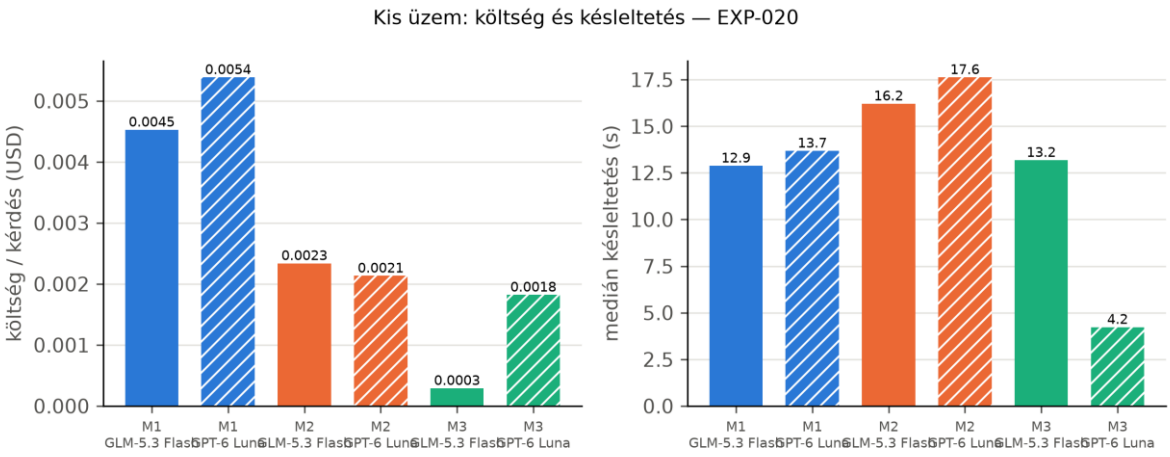
10. ábra: A végső teszt pontossága családonként, a hat párosításon.

A legjobb párosítás az M2 GLM-mel (31 helyes a 40-ből). GLM-mel az M1 szignifikánsan elmarad mind az M2-től (párosított különbség −0,150, 95% CI −0,275 és −0,025 között), mind az M3-tól (−0,125; −0,225 és −0,025 között), az M2 és az M3 különbsége viszont nem szignifikáns (+0,025; −0,075 és +0,125 között). Lunával egyik különbség sem szignifikáns. Egyik párosítás sem hallucinált a teszten; a megválaszolhatatlan kérdéseken (d) mindegyik módszer 9 vagy 10 helyes választ adott. A különbség a (b) és (c) családon jön ki: az adatlapi kérdéseken az M2 GLM-mel 7/10, az M1 GLM-mel 3/10.

Költség és késleltetés

15. táblázat: Költség, késleltetés és tokenhasználat kérdésenként a végső teszten

Módszer	Modell	Költség / kérdés (USD)	Medián késleltetés (s)	Átlagos bemeneti token / kérdés	Átlagos kimeneti token / kérdés
M1	GLM-5.3 Flash	0,00453	12,9	81 492	3 940
M2	GLM-5.3 Flash	0,00234	16,2	33 068	2 221
M3	GLM-5.3 Flash	0,00029	13,2	13 264	1 820
M1	GPT-6 Luna	0,00539	13,7	56 398	1 578
M2	GPT-6 Luna	0,00213	17,6	35 765	1 115
M3	GPT-6 Luna	0,00182	4,2	12 901	425

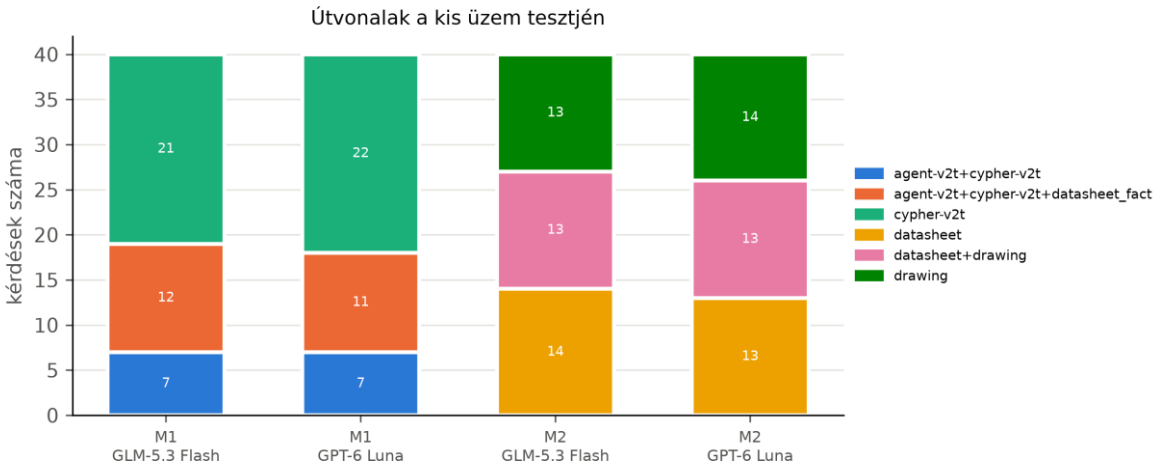


11. ábra: Kérdésenkénti költség és medián késleltetés; két külön panel, mert a két mennyiség skálája eltér.

A legdrágább az M1: a kaszkád első szintje a teljes kétrétegű sémát kapja promptban, és ha nem fogadják el a válaszát, az ágens is lefut, amely tag-enként külön adatlap-hívásokat indít. Az M2 GLM-mel körülbelül feleannyiba kerül (0,00234 vs. 0,00453 USD), Lunával 40%-ába (0,00213 vs. 0,00539 USD), mert csak a szükséges eszközt hívja. Az M3 a legolcsóbb, és Lunával a leggyorsabb is: egyetlen hívás, de a teljes rajzot és tizenkét adatlap-darabot viszi magával. Ez a példaüzemen olcsó, mert a rajz kicsi (36 csúcs); Benedek mérései szerint a ContextRAG költsége a lapszámmal meredeken nő (a valósághűbb generátor üzemein 1 lapon a kaszkád 3,79-szerese, 11 lapon 21,57-szerese, és kb. 62 lap fölött a rajz már nem fér a promptba), ezért az M3 előnye nagyobb üzemen nem várható.

Útvonalak

Az M1 GLM-mel a 40 tesztkérdésből 21-et a Cypher-szinten zárt le, 7-hez az ágens is kellett adatlap-hívás nélkül, 12-höz az ágens és legalább egy `datasheet_fact` hívás; Lunával ugyanez 22, 7 és 11. Az M2 GLM-mel 13 kérdésnél csak a rajzot, 14-nél csak az adatlapot, 13-nál mindkettőt kérdezte; Lunával 14, 13 és 13.



12. ábra: Az M1 és az M2 útvonalai a végső teszten.

Összetett, több adatlapos kérdések

A végső teszt után egy kiegészítő mérést is végeztünk 12 olyan kérdésen, amelyekhez a rajzon egy utat, egy elemhalmazt vagy két topológiai lépést kell bejárni, és két-három különböző adatlapot kell összevetni (például „Along the process flow path from E-1-1 to CHV-2-1, which item has the lowest maximum allowable pressure according to its datasheet?”). A referenciát program rakta össze a generált üzem pontos leírásából és egy nyelvi modell által a PDF-ekből gyűjtött tényekből; négy kérdésnél a helyes válasz az, hogy a rajz nem mondja meg a beépített változatot, ezért a legkisebb érték nem dönthető el. A kérdéseket futtatás előtt befagyasztottuk, és a módszereken semmit nem változtattunk.

16. táblázat: Az összetett, több adatlapos kérdések eredménye (12 kérdés)

Módszer	Modell	Helyes	Részben	Rossz	Hallucinált	Költség / 12 kérdés (USD)	Medián késleltetés (s)
M1	GLM-5.3 Flash	0	5	6	1	0,0636	61,9
M2	GLM-5.3 Flash	3	6	3	0	0,0537	34,3
M3	GLM-5.3 Flash	0	2	10	0	0,0046	25,6
M1	GPT-6 Luna	0	6	6	0	0,0880	28,2
M2	GPT-6 Luna	1	9	2	0	0,1280	38,6
M3	GPT-6 Luna	0	0	12	0	0,0276	13,6

Ezek a kérdések mindegyik módszernek nehezek: teljesen helyes választ csak a router adott (GLM-mel 3, Lunával 1 kérdésre), a bíráló pedig „részben helyes” ítéletet adott, ha egy elemnél a második érték vagy a feltétel hiányzott. Ha a részben helyes válaszokat is számítjuk, a router GLM-mel 9, Lunával 10 kérdésnél adott legalább részben jó választ, az M1 5, illetve 6, a gráf nélküli baseline 2, illetve 0 kérdésnél. A különbség itt a legnagyobb: több adatlap összevetéséhez a rajzon kell bejárni az elemeket, majd mindegyik adatlapját külön lekérdezni, és ezt a router eszközhívásai természetesen leképezik, az egyetlen lekérdezés (M1) és az egyetlen prompt (M3) viszont nem.

Hibaelemzés

Az M1 egyetlen lekérdezése. Az M1 hibáinak nagy része „részben helyes” válasz: a Cypher-szint egy szűk lekérdezéssel talál egy releváns adatpontot, a kaszkád elfogadja a nem üres eredményt, és a válaszból kimarad egy feltétel vagy egy második érték. A dev halmazon például a szivattyú 266 psi-s burkolatnyomása megvolt, a 100 °F-ig érvényes 250 psi-s üzemi határ nem. Az elfogadási szabály a rajzi kérdésekre készült, ahol a nem üres sorhalmaz jó jelzés; adatlapi kérdésnél a hiányos sor is nem üres.

A rajzi kérdések közös hibái. A teszt (a) családjában három kérdést több párosítás is elrontott. Az izolációs kérdésnél („To isolate P-1-5 from all upstream equipment, which valves must be closed?”) a referenciaválasz egyetlen szelep, a módszerek többsége a felfelé eső összes szelepet felsorolta. A szabályozókörös kérdésnél („Which control loops act on

valves directly connected to P-2-1?") az M2 mindkét modellel a szelepet (PV-2-1) adta meg a kör (PIC-2-1) helyett: a router a rajzeszköznek szabad szöveges kérdést ad, és a válasz alakja ott elcsúszott. A P-2-1-ből lefelé eső elemek halmazánál az M1 GLM-mel és az M3 Lunával hiányos listát adott.

Megválaszolhatatlan kérdések. GLM-mel mindhárom módszer mind a 10 kérdést felismerte, Lunával 9-et; hallucináció nem volt. A három Luna-hiba: az M3 egy nem létező tag-re (P-2-4) utat adott meg, az M2 egy nem létező tag-re (FV-2-3) üres listát adott a „nincs benne" jelzés nélkül, az M1 pedig egy adatlapban nem szereplő tényre jelzés nélkül válaszolt (részben helyes).

A referencia korlátja. A (b)–(d) család referenciaválaszait egy modell írta a PDF-ből, és ember nem ellenőrizte őket; a bíráló ugyanazt a referenciát látta minden módszernél, ezért az összevetés igazságos, de az abszolút számok a referencia hibáival együtt értendők.